

The image shows a well-lit industrial networking lab. In the foreground, there is a long wooden workbench on a black frame with many drawers. A black ergonomic chair is positioned in front of it. In the background, several rows of black equipment racks are filled with various electronic components and cables. A projector is mounted on the ceiling, and the room has a drop ceiling with fluorescent lights.

INDUSTRIAL NETWORKING MENG 4010

CONTENTS

9A. EtherCAT Network

- 9A.1 The Evolution of Industrial Communication
- 9A.2 Introduction to EtherCAT Fundamentals
- 9A.3 EtherCAT Protocol Deep Dive
- 9A.4 EtherCAT Diagnostics
- 9A.5 EtherCAT Communications Profile
- 9A.6 EtherCAT Redundancy

9A.1 The Evolution of Industrial Communication

- 1960s-70s: Hardwired Systems

Early automation relied on complex, point-to-point wiring to relays and PLCs. This was inflexible, as any system change required physical rewiring.

- 1970s-80s: Serial Communication (RS-232/422/485)

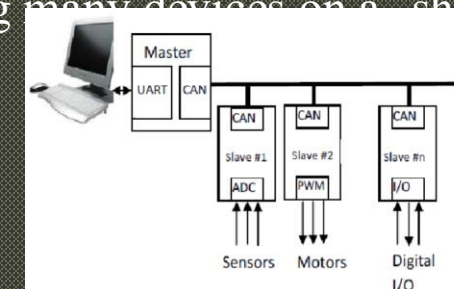
Digital serial links replaced some wiring, allowing multiple devices to connect. However, they were slow, prone to delays, and lacked standardized protocols.

Specifications	RS-232	RS-422	RS-485
Cabling	Point-to-Point	Point-to-Point Multi-drop	Point-to-Point Multi-drop
Number of Devices	1 transmitter 1 receiver	1 transmitter 10 receivers	32 transmitters 32 receivers
Max. Theoretical Distance	50 feet	4000 feet	4000 feet
Max. Data Rate	1Mbps	10Mbps	>10Mbps
Duplex Type	Full Duplex	Full Duplex Half Duplex	Full Duplex Half Duplex
Operation	Unbalanced	Balanced (Differential)	Balanced (Differential)
Receiver Sensitivity	± 3 mV	± 200 mV	± 200 mV

<https://oringnet.com/en/knowledge-base/what-is-rs-232,-rs-422-and-rs-485>

- 1980s-90s: Fieldbus Systems (PROFIBUS, Modbus, DeviceNet)

Dedicated digital networks (Fieldbuses) dramatically reduced cabling by connecting many devices on a shared bus. They offered faster, deterministic (real-time) communication for control tasks.



- Benefits: Simplified installation, easier maintenance, and vendor interoperability.

- Limitations: Limited bandwidth, proprietary options caused compatibility issues, and integration with IT networks³.

https://www.researchgate.net/figure/Overall-CAN-fieldbus-master-slave-mode_fig1_340318168

9A.1 The Evolution of Industrial Communication

Transition to Ethernet-Based Industrial Networks

- Traditional Ethernet (IEEE 802.3) originally used CSMA/CD (Carrier Sense Multiple Access with Collision Detection).
 - CSMA/CD worked well in offices but introduced unpredictable delays → not deterministic.
- Industrial automation required deterministic timing for motion and control loops.
 - Solution: Industrial Ethernet protocols modified Ethernet to guarantee real-time communication.
- Full-duplex switched Ethernet removed collisions, enabling consistent timing.
- Industrial Ethernet uses Layer 2 and 3 enhancements for synchronization and deterministic scheduling.
- Common Industrial Ethernet protocols:
 - PROFINET: Developed by Siemens; uses RT (Real-Time) and IRT (Isochronous Real-Time) modes.
 - EtherNet/IP: Developed by Rockwell Automation; uses standard Ethernet/TCP/IP with CIP objects.
 - EtherCAT: Developed by Beckhoff Automation; uses 'on-the-fly' processing for extreme speed.
 - Modbus TCP: Simple and open Ethernet variant of Modbus RTU.
 - POWERLINK: Developed by B&R; deterministic scheduling with polling mechanism.

ORGANIZATION	RESPONSE TIME (for 100 axes)	JITTER	DATA RATE
Ethernet/IP CIPSync ODVA	1ms	<1ms	100Mbit/s
Ethernet Powerlink EPSP	<1ms	<1ms	100Mbit/s
PROFINET-IRT PNO	<1ms	<1ms	100Mbit/s
SERCOS-III IGS	<0.5ms	<0.1ms	100Mbit/s
EtherCAT ETG	0.1ms	<0.1ms	100Mbit/s

Real-Time Communication Levels

- Soft Real-Time: Delays tolerated (e.g., HMI updates, logging).
- Hard Real-Time: No delay tolerated (e.g., servo synchronization, motion control).
- EtherCAT and PROFINET IRT belong to hard real-time category.

9A.1 The Evolution of Industrial Communication

To make Ethernet work in real-time applications (like industrial control), different methods are used to improve timing and reliability:

- Turn off CSMA/CD (the standard way Ethernet handles traffic) and replace it with time slots or polling.
- Use special switches that control when and how fast data is sent.

Even with these solutions, the speed of sending data to outputs or reading inputs still depends on how the system is built and programmed.

- CSMA/CD (used in regular Ethernet) lets devices take turns sending data, but if two send at once, a collision happens. Fixing this takes time, so it is not good for real-time systems.
- Time-slotted and Polling: Industrial networks often use assigned time slots, or a master device polls each one in order, which helps control timing better.
- EtherCAT: Sends one message through all devices, and each device reads and writes its data on the fly. This makes communication very fast and synchronized.
- PROFINET IRT: Also uses precise scheduling managed by a master, ensuring time-critical data always arrives on time.

		EtherNet/IP [™]	EtherCAT [™]
Stack Time	Profinet IO	Ethernet/IP	EtherCAT
Average	0.58 ms	1.89 ms	0.11 ms
Max.	0.74 ms	2.96 ms	0.18 ms
Min.	0.54 ms	1.23 ms	0.05 ms

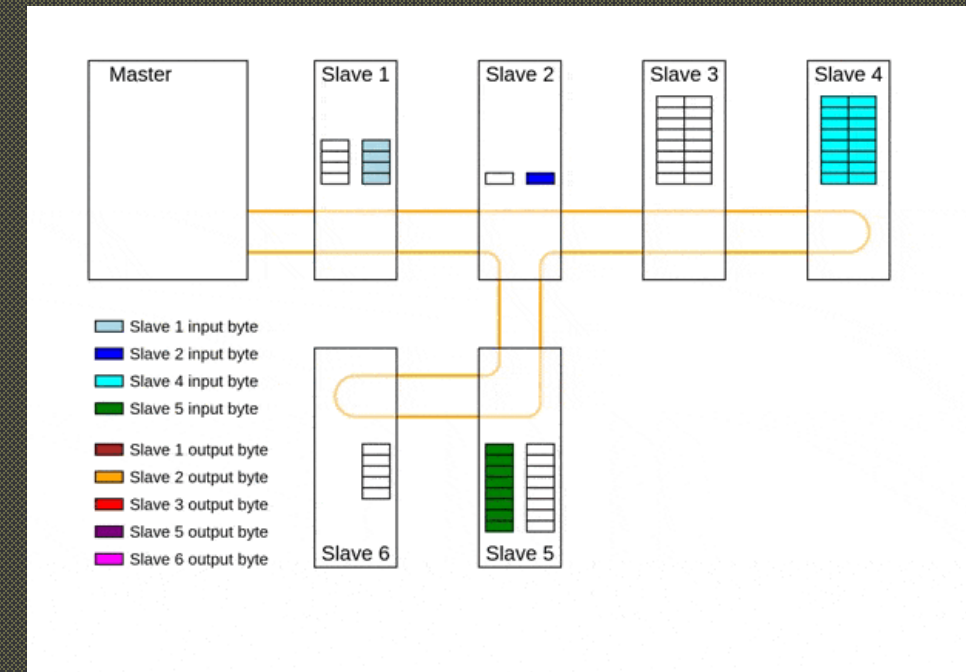
9A.1 The Evolution of Industrial Communication

EtherCAT = Ethernet for Control Automation Technology (Beckhoff, early 2000s).

- Managed by EtherCAT Technology Group (ETG) - open standard, global adoption.
- Processes data while the Ethernet frame passes through each node ('on-the-fly' processing).
- Provides ultra-fast communication (cycle times < 100 μ s).
- Synchronization accuracy: < 1 μ s jitter using distributed clocks.
- Supports thousands of devices with minimal delay.

Typical Applications

- Robotics and motion control systems.
- CNC machines with multiple servo axes.
- Packaging, printing, and bottling lines.
- Renewable energy systems (wind turbines, solar inverters).
- Smart grid monitoring and data acquisition.



9A.1 The Evolution of Industrial Communication

The EtherCAT features include the following.

Ultra-Fast I/O Response Time

- Operates at 100 Mbps full-duplex
- Enables very short delays between input detection and output action
- Ideal for motion control and real-time feedback

High Communication Efficiency

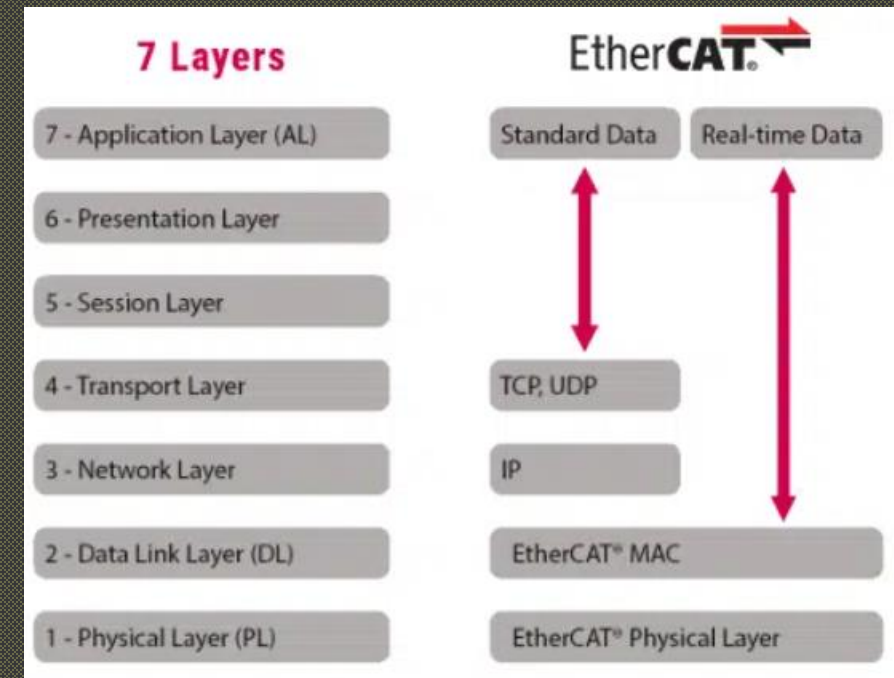
- Uses a single shared Ethernet frame to transfer all I/O data
- On-the-fly data processing ensures minimal latency
- Bandwidth is fully utilized - very low protocol overhead

Full Compatibility with Standard Ethernet

- Works with standard Ethernet cables, connectors, and switches
- No special physical layer or wiring required
- Supports existing Ethernet diagnostic tools

Open and Vendor-Neutral Technology

- EtherCAT is not proprietary - supported by multiple vendors
- Open standard maintained by the EtherCAT Technology Group (ETG)



<https://www.kebamerica.com/blog/5-advantages-of-using-ethercat-for-motion-control/>

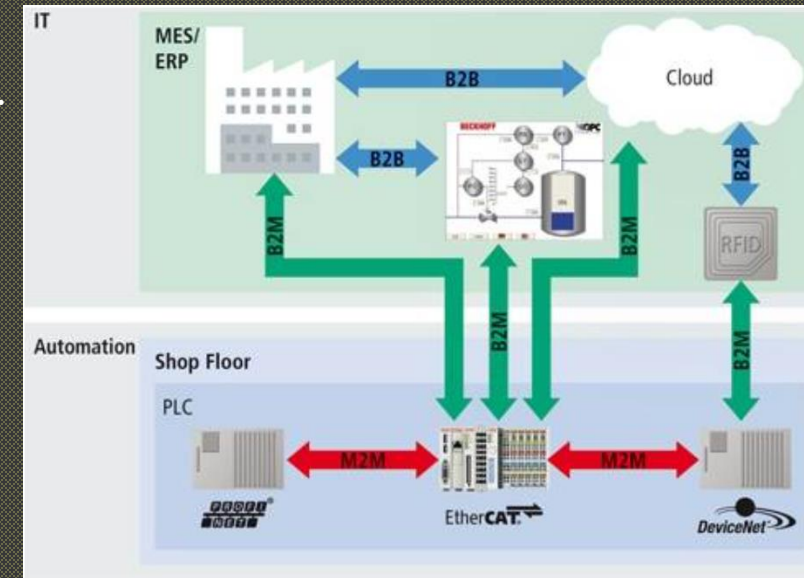
9A.1 The Evolution of Industrial Communication

Advantages of Ethernet-Based Systems

- Performance advantages:
 - 100 Mbps or higher bandwidth.
 - Low latency and deterministic operation.
- Maintenance advantages:
 - Easier fault detection using diagnostic tools (e.g., Wireshark, TwinCAT).
 - Remote access and configuration via standard TCP/IP.
- Scalability advantages:
 - Supports flexible topologies (line, ring, star, tree).
 - Easily integrates new devices without major reconfiguration.
- Cost and integration advantages:
 - Uses standard Ethernet cables and switches.
 - Simplifies IT/OT integration and data exchange.

Industry 4.0 and Ethernet Convergence

- Industry 4.0 emphasizes data integration across all production levels.
- EtherCAT contributes by enabling deterministic data flow at the machine level.
- Modern systems combine EtherCAT for control + OPC UA/MQTT for cloud data exchange.
- Edge devices and gateways bridge control networks and IT infrastructure.
- Cybersecurity considerations: VLANs, firewalls, and network segmentation are critical.



<https://www.controleng.com/forerunner-to-industry-4-0-and-the-internet-of-things/>

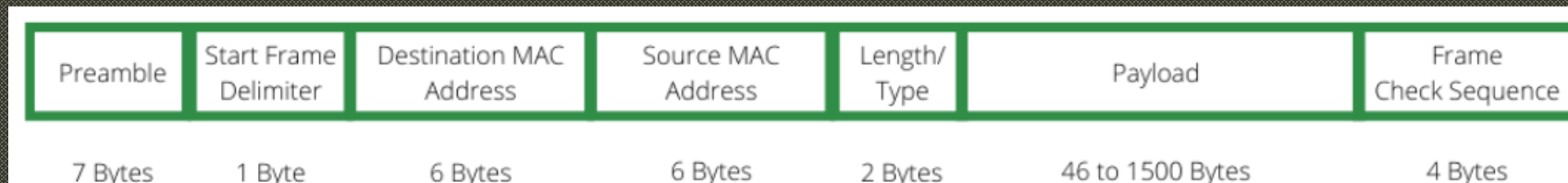
9A.1 The Evolution of Industrial Communication

Example: Imagine a motor drive is sending 4 bytes of its current data (like speed or status) and receiving 4 bytes of control data (like set speed or commands) - this exchange happens regularly and quickly.

Now, in real-time Ethernet systems, each exchange typically happens inside a full Ethernet frame. But here is the issue:

- A full Ethernet frame is 84 bytes, even though we are only sending 4 bytes of useful data.
- So, only 4 out of 84 bytes are useful payload, and the rest is overhead (frame headers, addressing, etc.).
- This means that only about 4.8% of the network's capacity is used for real data, even under perfect conditions (100% bus load, no delays at all).

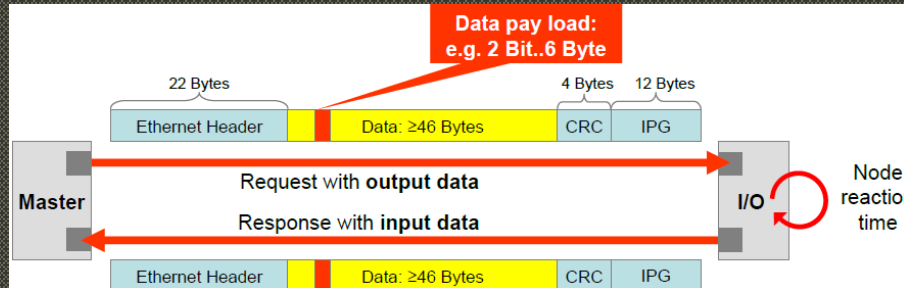
If there is a tiny delay - for example, the drive takes 10 microseconds to respond - that small delay reduces the efficiency even more, down to just 1.9% usable data rate.



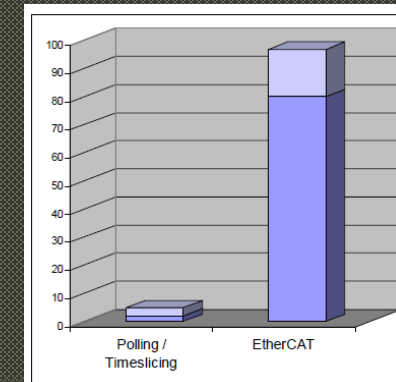
Minimum Frame Size: 64 bytes.
Preamble: 7 bytes.
Start Frame Delimiter (SFD): 1 byte.
Interpacket Gap (IPG): 12 bytes.
Total: $64 + 7 + 1 + 12 = 84$ bytes.

9A.1 The Evolution of Industrial Communication

- In the example, the drive is sending and receiving 4 bytes of useful data within an 84-byte Ethernet frame.
So, efficiency = useful data / total data per frame:
 - $Efficiency = 4 / 84 = 4.76\% \approx 4.8\%$This assumes no time lost - meaning the drive responds instantly, and the next frame can be sent right away.
- If the drive takes 10 microseconds (μs) to respond before the next frame can be sent, we now have to wait 10 μs between transmissions.
 - Let us say it takes about 6.72 μs to transmit one 84-byte frame over a 100 Mbps Ethernet link
because: $84 \times 8 \text{ bits} = 672 \text{ bits}$, $(672 \text{ bits}) / (100 \times 10^6) \text{ bits/sec} = 6.72 \mu s$
 - Adding 10 μs of response delay to each transmission cycle, each data exchange (transmit + wait) takes:
 - $6.72 \mu s + 10 \mu s = 16.72 \mu s$
 - But the useful payload is still just 4 bytes, so now we calculate the efficiency per unit of time:
 - $Efficiency = \frac{\text{Useful Bits}}{\text{Total Data Frame}} = \frac{4 * 8}{16.72 \mu s * 100 * 10^6} = 1.91\% \approx 1.9\%$



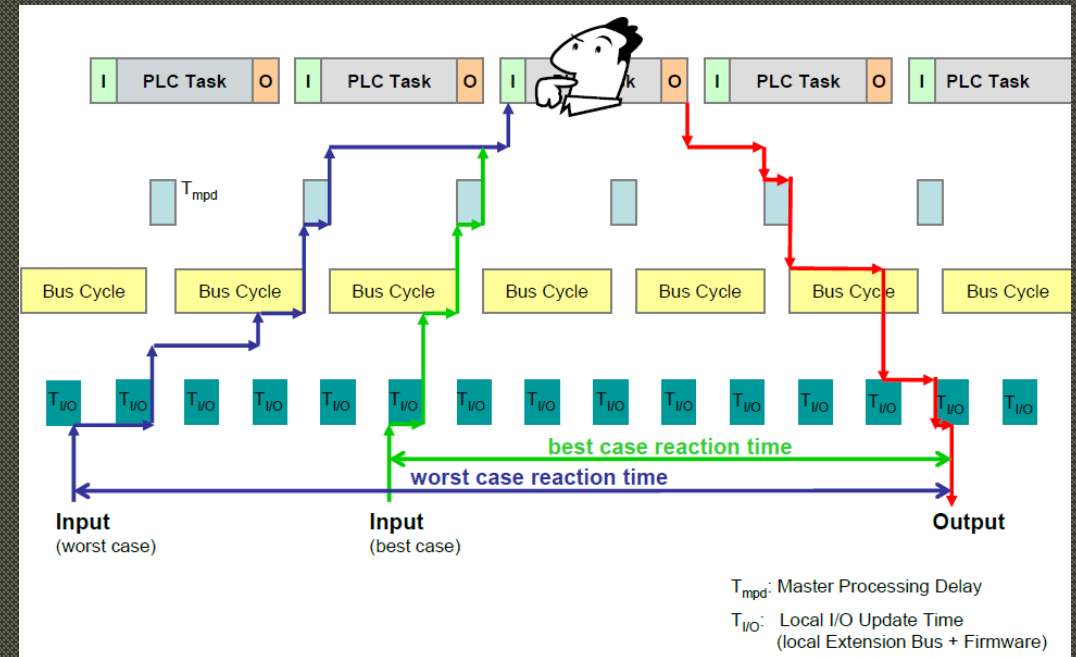
https://www.etherncat.org/2011/us/seminar_download/files/EtherCAT_US_2011_03_TechnologyOverview.pdf



9A.1 The Evolution of Industrial Communication

Reaction time (response delay) in conventional fieldbus systems

Stage	Best Case (Quick Path)	Worst Case (Slow Path)	Why the Difference?
Input Detection	T_{IO} + immediate bus cycle	T_{IO} + full extra bus cycle wait	Timing of arrival vs. cycle end
To PLC	1 bus cycle + 1 PLC task + 1 T_{mpd}	2-3 bus cycles + 2 PLC tasks + 2 T_{mpd}	Missed handoffs (like missing a train)
Processing	Minimal T_{mpd} (CPU fetches inputs from memory to logic + quick computations)	Wait for next task + extra T_{mpd} (additional CPU fetches/updates)	PLC runs in fixed loops; logic execution adds CPU delay
To Output	1 bus cycle + T_{IO}	2 bus cycles + T_{IO}	Same bus congestion on return
Total Time	Short (e.g., 2-3 cycles total)	Long (e.g., 5-7 cycles total)	Cumulative waits add up



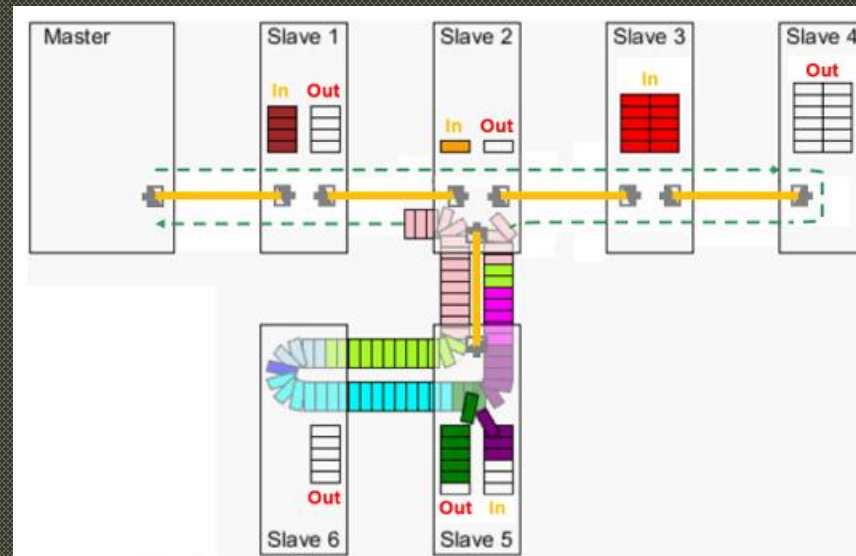
EtherCAT Technology Group, 2009

- **PLC Task (I/O):** Is the time the CPU reads the inputs, executes the logic, writes the outputs, and does the diagnostics.
- **T_{mpd} (Master Delay):** Represents the PLC's time to get the status of the inputs from the memory to the logic and sent the status of the output from the logic to the memory.
- **Bus Cycle:** A repeating time interval (like every 10-100 milliseconds) where the fieldbus network sends and receives data from I/O devices all at once.
- **T_{IO} (Local I/O Update Time):** Represents the time taken by an I/O device to read or write its local data.

9A.2 Introduction to EtherCAT Fundamentals



- EtherCAT is designed for real-time industrial automation, motion control, and high-speed I/O.
- Essential principle: send one Ethernet frame through all devices; each device reads/writes on the fly.
- Key advantages:
 - Ultra-fast communication (~100 Mbps full-duplex)
 - High efficiency (>90% usable bandwidth)
 - Precise synchronization (<1 μ s)
 - Works with standard Ethernet cables and switches
- Real-world analogy: Think of a single envelope (Ethernet frame) traveling along a chain of offices (devices). Each office opens it, inserts its memo, and closes it - without slowing down the envelope.



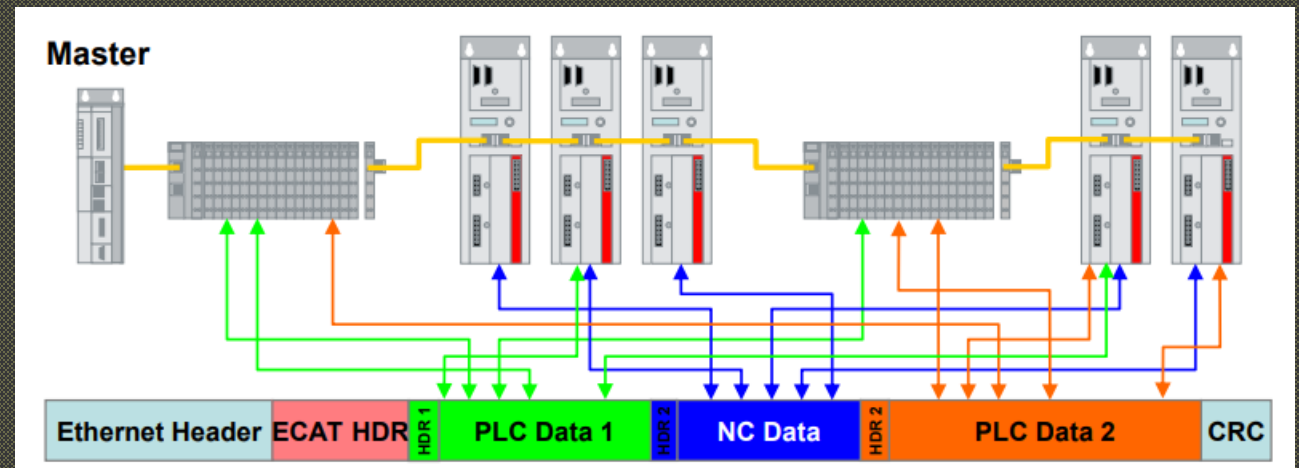
www.ethercat.org

- Master-Controlled: The master initiates all communication.
- Single Frame Loop: One frame visits all slaves and returns to the master.
- Topology Independent: The logical path is always a loop, even if the physical wiring isn't.
- On-the-Fly Processing: Slaves process data in real-time as the frame passes through them, which is a key feature for its high speed.

9A.2 The Introduction to EtherCAT Fundamentals

EtherCAT Frame Structure

- EtherCAT builds on standard Ethernet frames.
- Uses EtherType 0x88A4 to identify an EtherCAT frame.
- Frame components:
 - Ethernet header (MAC addresses, type)
 - EtherCAT header
 - 1-15 datagrams (each containing datagram header, payload, working counter)
 - Frame Check Sequence (FCS)
- Minimum frame size: 64 bytes; padded if smaller
- Example:
 - Device sends 4 bytes of control data
 - Standard Ethernet frame: 84 bytes
(64 + 7 preamble + 1 SFD + 12 IPG)
 - Efficiency with standard Ethernet: ~4.8%
 - Efficiency with EtherCAT: ~90%
(due to on-the-fly processing)

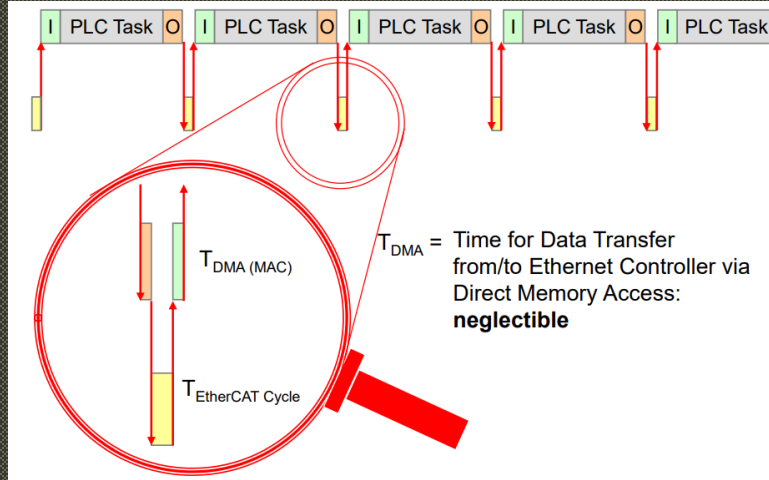


https://www.ethercat.org/download/documents/EtherCAT_Introduction_EN.pdf

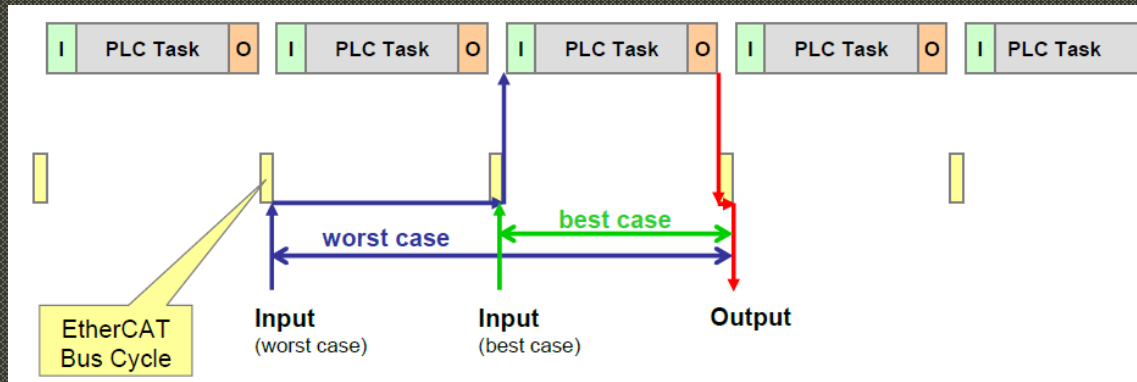
Interframe Gap (IPG) 12 (minimum) - an idle period required between the transmission of consecutive frames.

9A.2 The Introduction to EtherCAT Fundamentals

Reaction time with EtherCAT



- The PLC tasks remain the same as before. EtherCAT does not require changes to your logic or controller performance.
- The EtherCAT bus cycle replaces the old, slower fieldbus cycle.
- Data transfer between the EtherCAT controller (master) and network interface happens through Direct Memory Access (DMA), meaning data moves directly without the CPU needing to copy it.
- T_{DMA} (MAC) = time needed for the data to move from memory to Ethernet hardware → it is very small (almost negligible).
- Result: Inputs and outputs are updated almost instantly because data passes “on the fly” through the EtherCAT frame.



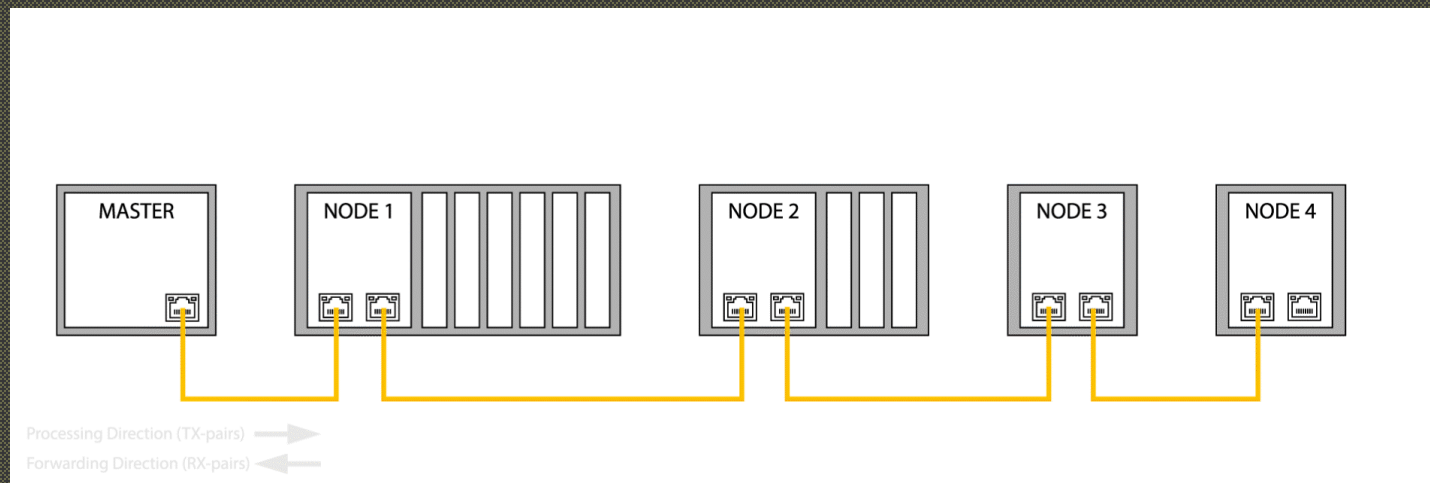
- EtherCAT makes the input-to-output reaction very fast.
- The gap between worst case and best case is much smaller than in legacy systems.
- There are no extra local I/O or bus cycles slowing things down.
- The EtherCAT frame travels through each device, updating I/O data on the fly in the same communication cycle.

9A.2 The Introduction to EtherCAT Fundamentals

On-the-Fly Processing

- EtherCAT uses EtherCAT Slave Controllers (ESCs) in each device.
- Each ESC reads/writes data while the frame passes, avoiding full reception/retransmission.
- Hardware delay per device: a few nanoseconds
- Benefits:
 - Eliminates delays from local I/O cycles
 - No need for special master hardware
 - Allows very short bus cycles ($\leq 100 \mu\text{s}$)

Analogy: Like a train passing through stations; each station adds its cargo without stopping the train.

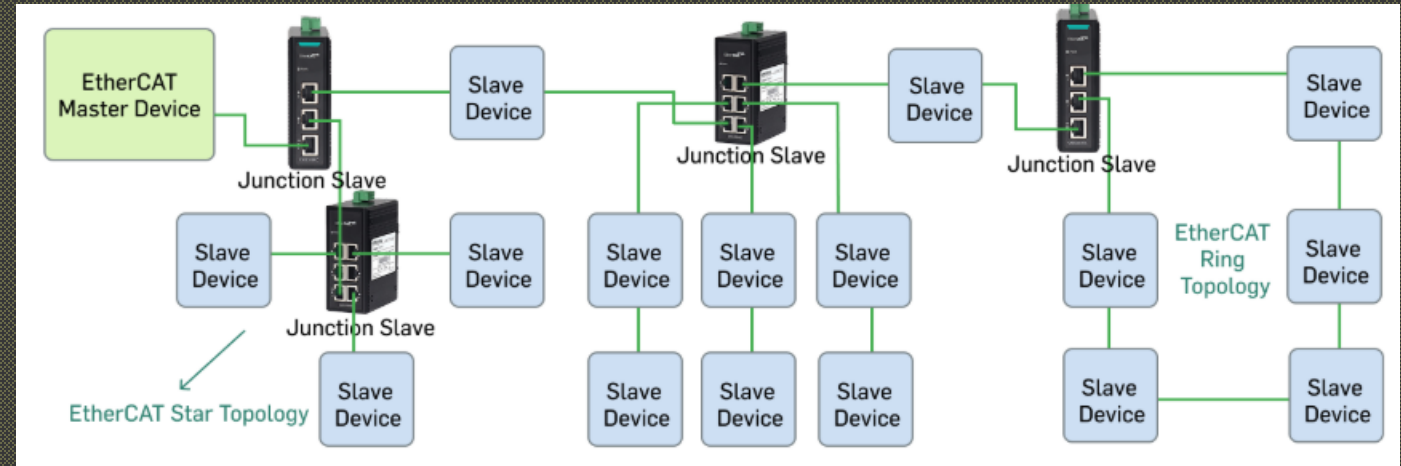


9A.2 The Introduction to EtherCAT Fundamentals



EtherCAT Device Types

1. MainDevice (Master/Controller)
 - Sends and receives frames
 - Can run on a standard PC or PLC
 - Handles process data image
 - CPU load is low (uses DMA)
2. SubDevice (Slave) – Ethernet Slave Controller
 - Connects sensors, actuators, I/O modules
 - Can use ASIC, FPGA, or low-cost microcontroller
 - Stores basic info in EEPROM/SII
 - Handles on-the-fly reading/writing
3. Couplers and Terminals
 - Expand EtherCAT networks
 - EtherCAT P terminals provide power + data over single cable
 - Real-world example:
 - A Beckhoff servo drive acts as a SubDevice.
 - Master sends setpoint data; servo reads on the fly, updates motor position; status returns in the same frame.¹⁶



<https://www.come-star.com/blog/ethercat-network-topology/>

ASIC - Application-Specific Integrated Circuit

FPGA - Field-Programmable Gate Array

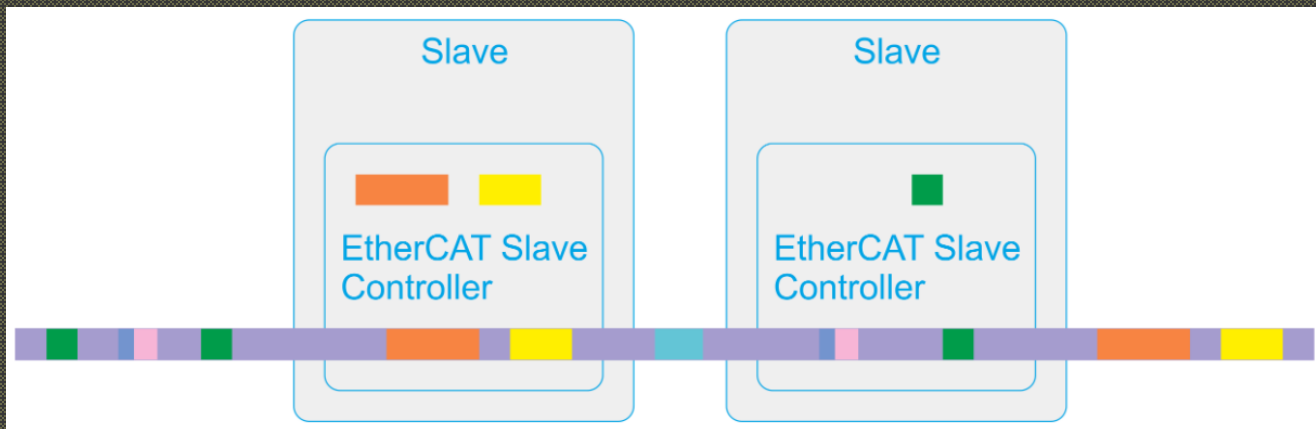
EEPROM - Electrically Erasable Programmable Read-Only Memory

SII - Slave Information Interface

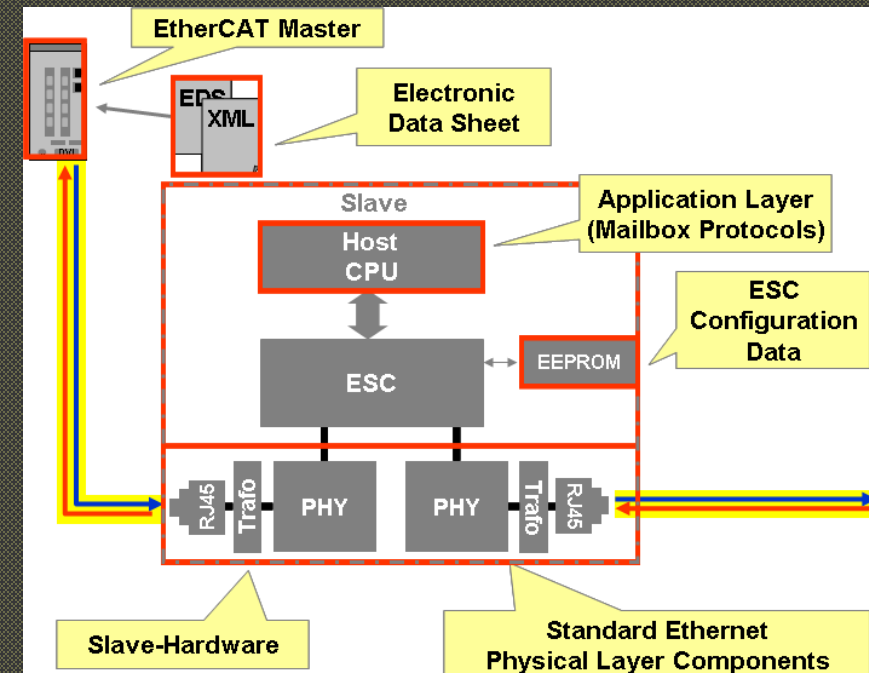
9A.2 The Introduction to EtherCAT Fundamentals

The EtherCAT Slave Controller (ESC)

- EtherCAT devices interact with passing frames in real time through a narrow data window, allowing them to read or write individual bits or larger data packets on the fly.
- Up to 65,535 devices can be connected in various topologies such as line, tree, star, or ring (for redundancy), offering flexibility in network layout.



https://infosys.beckhoff.com/english.php?content=../content/1033/tc3_io_intro/1257993099.html&id=



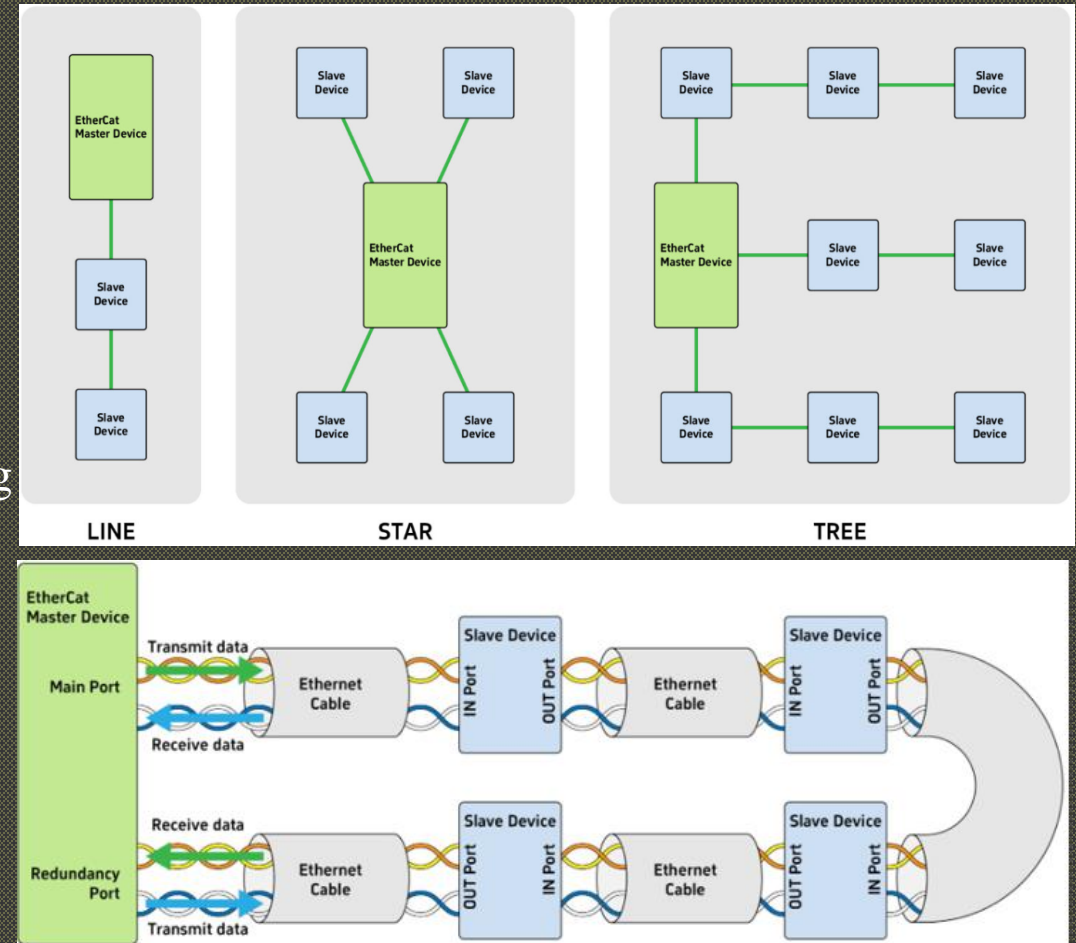
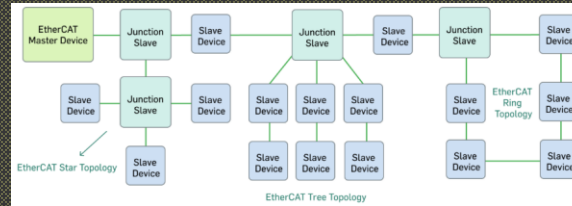
17

<https://infosys.beckhoff.com/english.php?content=../content/1033/ethercatsystem/2469118347.html&id=>

9A.2 The Introduction to EtherCAT Fundamentals

Topology and Network Layouts

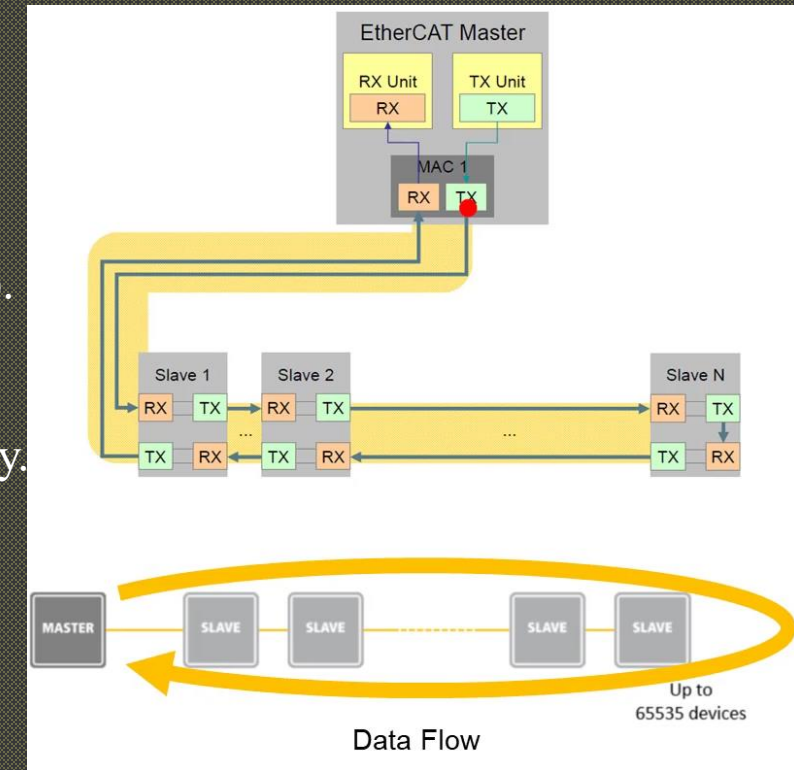
- Supported topologies:
 - Line (most common)
 - Tree
 - Star
 - Ring (redundancy)
 - Hot Connect: Add/remove nodes during operation without stopping network ($<15 \mu\text{s}$)
 - Max nodes per segment: 65,535
 - Cable types:
 - Standard Ethernet (Cat5/6, up to 100 m)
 - EtherCAT P (data + power)
 - Fiber optics (long distances)
- Design tip: Daisy-chain topology reduces cabling, simplifies wiring, and maximizes real-time performance.



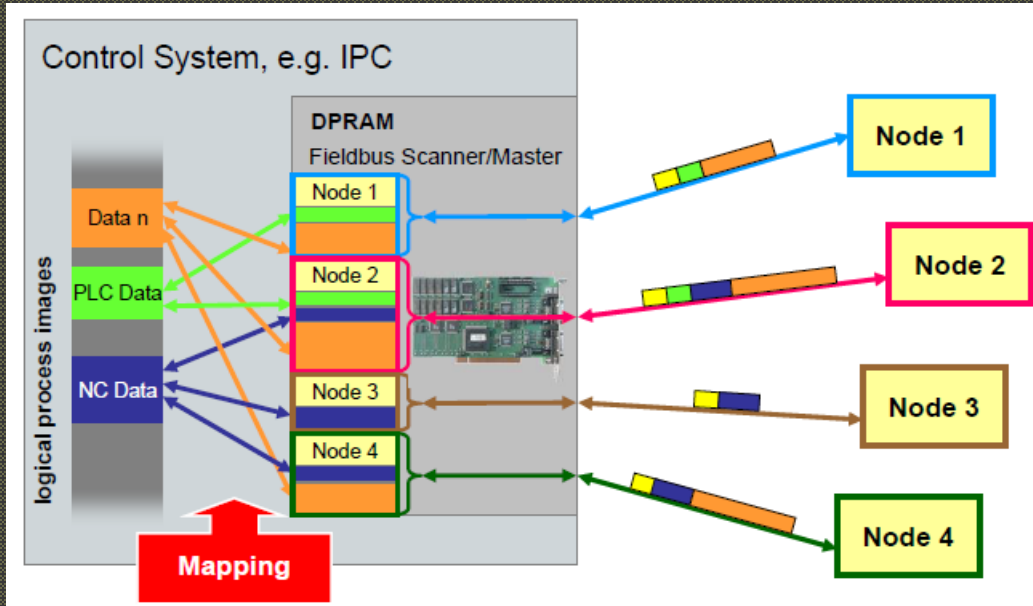
9A.2 The Introduction to EtherCAT Fundamentals

EtherCAT can create long bus structures with hundreds of nodes without needing switches or hubs.

- Drop lines can be added easily using built-in ports on many I/O modules.
- Hot Connect allows nodes or segments to be added or removed during operation without interrupting the rest of the network, thanks to auto-closing ports and fast detection ($<15\ \mu\text{s}$).
- EtherCAT works with standard Ethernet cables (up to 100 m between nodes), EtherCAT P (data + power over one cable), and fiber optics (for longer distances).
- Up to 65,535 nodes can connect to one EtherCAT segment.
- Devices like I/O slices can each be a full EtherCAT node, eliminating the need for local extension buses and increasing speed and simplicity.

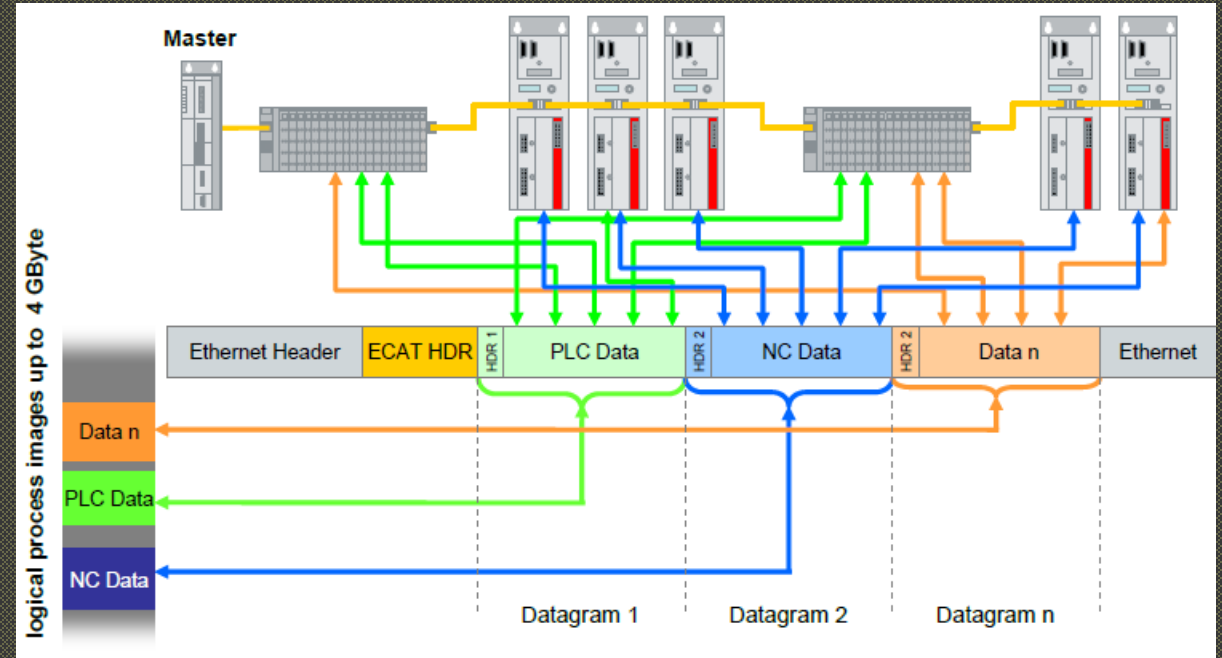


9A.2 The Introduction to EtherCAT Fundamentals



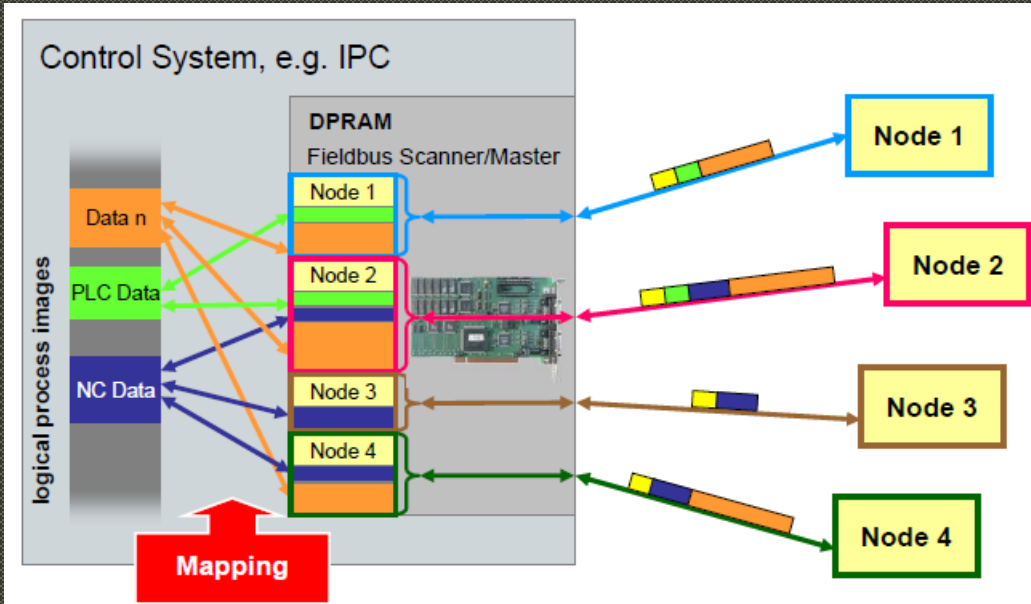
EtherCAT Technology Group, 2009

- The Control System (e.g., IPC) contains a Fieldbus master with DPRAM.
- All devices (Node 1, Node 2, Node 3, Node 4) send their data into the master's process image.
- The process image is a big memory area that stores all I/O data for the control system.
- However, this data must be mapped, meaning it has to be organized and assigned to the right place in the control system's memory.
- This mapping process is manual and time-consuming.

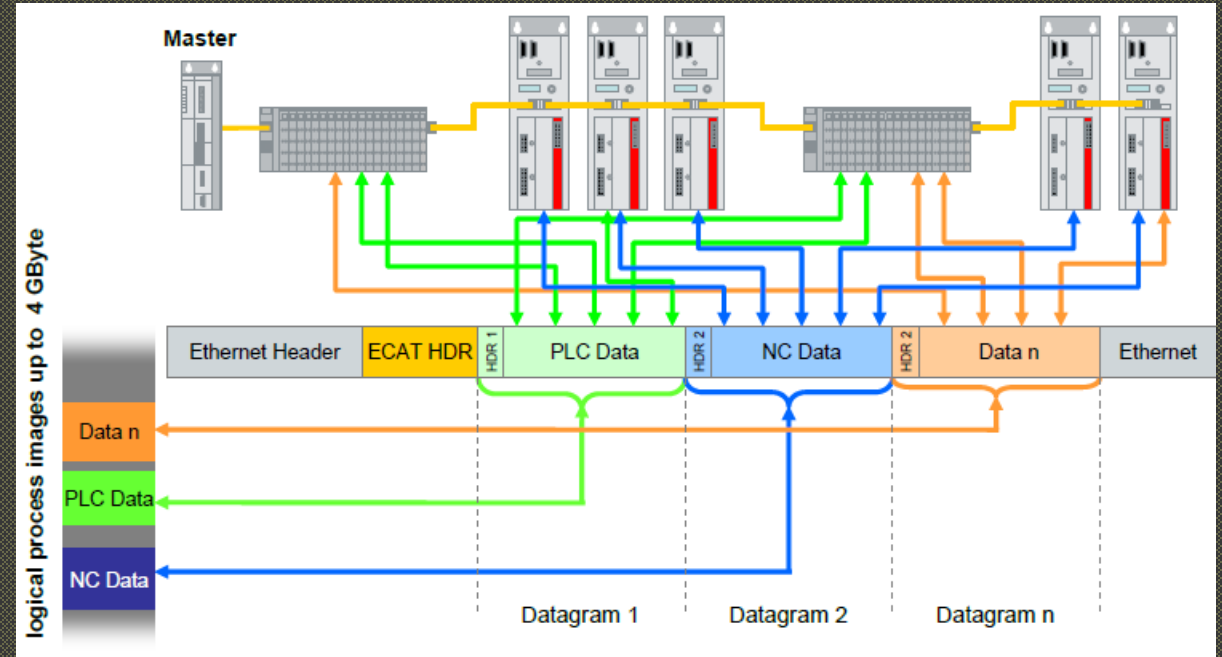


- In EtherCAT, mapping is moved inside the slave devices instead of being handled by the master.
- The master just sends a single EtherCAT frame that passes through each slave.
- Each slave reads (or writes) only the data meant for it directly within the same frame - this is called on-the-fly processing.
- The result is a single, logical process image that's automatically organized across all slaves.

9A.2 The Introduction to EtherCAT Fundamentals



EtherCAT Technology Group, 2009



Drawbacks

- The master must handle all mapping work, increasing CPU load.
- Any changes in device configuration require remapping.
- Communication and updates are slower because data goes through multiple layers.

- AdvantagesThe control system (master) is simpler and has less computational load.
- Data transfer is faster and more efficient since there's no separate mapping step.
- Communication is synchronized and tailored to each device's needs.

Feature	Fieldbus	EtherCAT
Mapping Location	In the control system (master)	In the slave devices
Master Load	High – must manage all mapping	Low – simplified control system
Speed	Slower (extra mapping step)	Faster (on-the-fly processing)
Flexibility	Limited – manual mapping	High – automatic mapping
Data Handling	Physical process image must be mapped	Logical process image built automatically

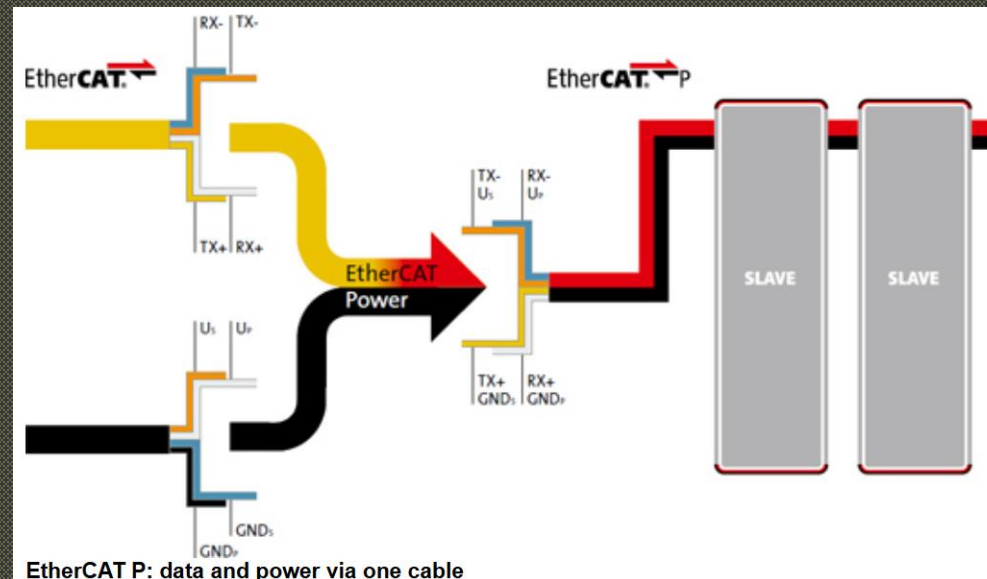
9A.2 The Introduction to EtherCAT Fundamentals

EtherCAT P is an extension of the EtherCAT standard that combines data and power transmission over a single four-wire Ethernet cable. It maintains the same protocol and communication advantages as EtherCAT, but enhances the Physical Layer to deliver power alongside data.

- No protocol change: Uses the same EtherCAT SubDevice Controllers.
- Two power lines integrated with data: US for system/sensors and UP for actuators/peripherals
- Reduced cabling: Fewer cables and connectors save space, cost, and simplify design.
- Ideal for sensors and modular machine parts: Especially useful in remote or isolated components.
- Single M8 connector simplifies integration and prevents misconnection through mechanical coding.
- Coexistence with standard EtherCAT: Can be used in the same network with rectifier units maintaining compatibility.

US and UP are two separate 24V power lines in EtherCAT P:

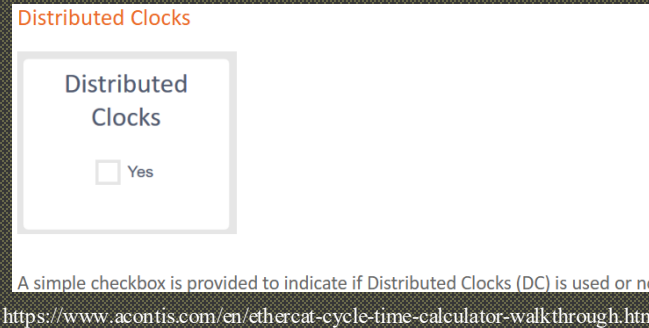
- US (Sensor/System Supply): Powers sensors and control electronics.
 - UP (Periphery/Actuator Supply): Powers high-load devices like motors and valves.
- They run over the same cable but are isolated to protect sensitive components.



9A.2 The Introduction to EtherCAT Fundamentals

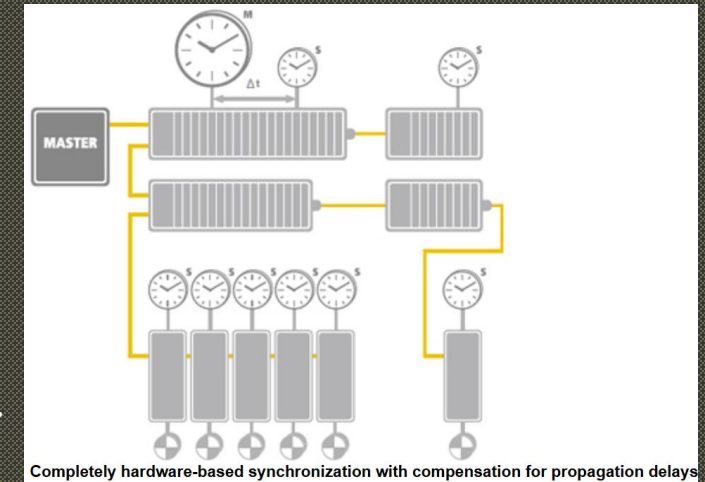


- EtherCAT ensures deterministic data communication, meaning every data exchange happens in a predictable, precisely timed manner.
- The key technologies that make this possible are:
 - On-the-fly frame processing
 - Distributed Clocks (DC)
 - Efficient bus cycle management



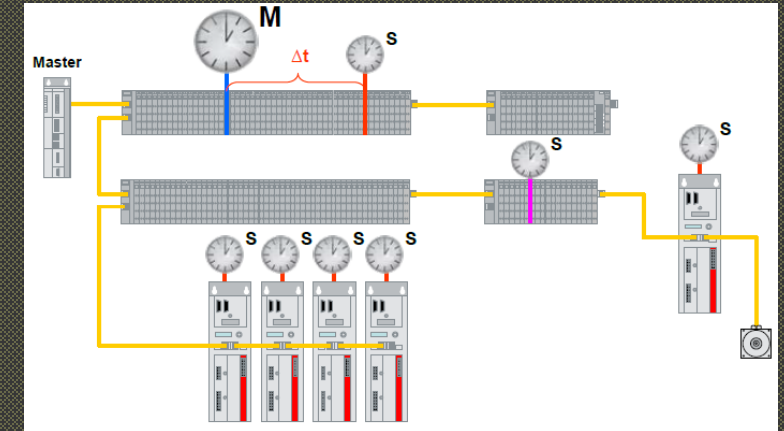
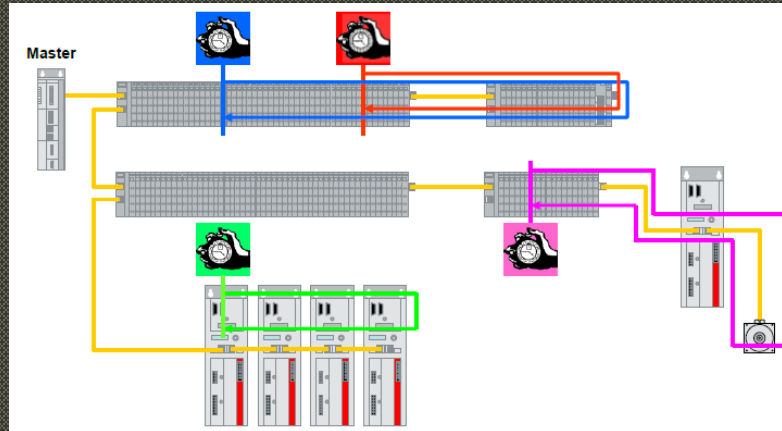
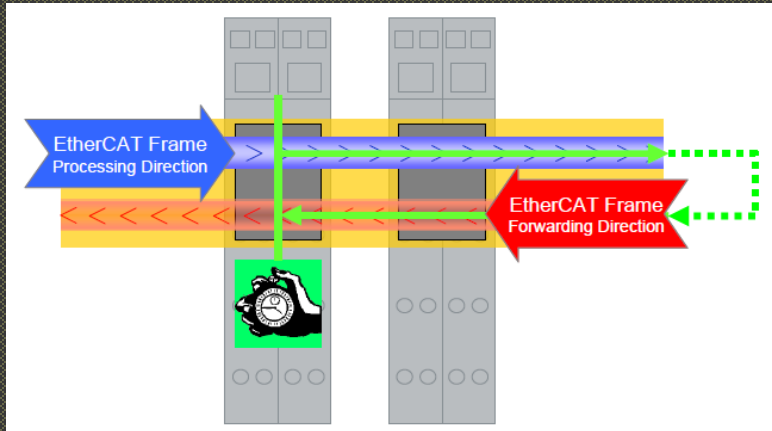
Distributed Clocks (DC)

- Purpose: To synchronize all EtherCAT devices (slaves) to a common, precise time base.
- How it works:
 - One device (usually the first EtherCAT slave) automatically becomes the reference clock.
 - All other slaves adjust their local internal clocks by continuously comparing and compensating for any drift relative to this reference.
 - The synchronization accuracy is better than 1 microsecond (μs) - often around 100 nanoseconds in practice.
- Result: Every device on the network shares the same notion of time.
- Applications:
 - Ideal for multi-axis servo systems (e.g., robotic arms, CNC machines, synchronized conveyors).
 - Ensures that motors and actuators start, move, and stop at exactly the same moment.
 - Enables time-stamped data acquisition, essential for diagnostics and precise motion tracking.



The EtherCAT - Synchronization

- EtherCAT achieves precise time synchronization by:
 - Measuring how long frames take to travel between devices.
 - Automatically adjusting internal clocks.
 - Keeping all nodes aligned with the master clock.
- Result: Synchronization accuracy better than 1 microsecond ($<1\ \mu\text{s}$) - essential for applications like motion control, robotics, and high-speed automation.



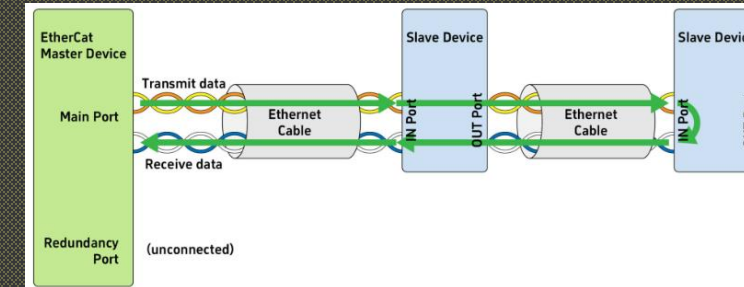
Concept	Description
Distributed Clocks (DC)	System that synchronizes all device clocks in an EtherCAT network.
Reference Clock	One device acts as the master time reference.
Time Measurement	Each node measures frame travel time to calculate delay.
Synchronization Accuracy	Better than 1 microsecond.
Benefit	Perfectly coordinated motion and data timing across all devices.

EtherCAT Technology Group, 2009

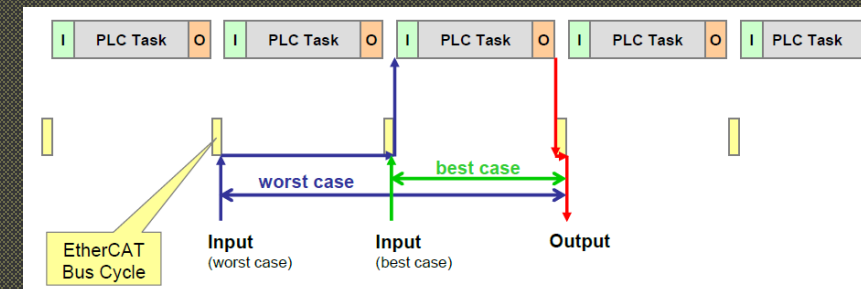
9A.2 The Introduction to EtherCAT Fundamentals



- Bus Cycle:
 - Represents one complete EtherCAT frame traveling from the master through all slaves and back.
 - Defines how frequently data is exchanged between master and slaves.
 - Typical EtherCAT bus cycle times range from 50 μ s to 1 ms, depending on system configuration.
- Reaction Time:
 - The time between an input event (e.g., sensor detection) and the corresponding output response (e.g., motor activation).
 - EtherCAT minimizes reaction time because data is processed immediately as it passes each device, not after the entire frame completes.
- Scenarios:
 - Best Case:
 - Input change occurs just before the master's task cycle
→ processed immediately in the next control loop.
→ total delay $\approx$ 1 bus cycle.
 - Worst Case:
 - Input change occurs just after a bus cycle
→ must wait for the next EtherCAT cycle.
→ total delay $\approx$ 2 bus cycles.
- Key Advantage:
 - EtherCAT eliminates additional delays from local I/O controllers or sub-bus systems, unlike legacy fieldbus networks.
 - This ensures highly deterministic and repeatable system behavior.



<https://www.come-star.com/blog/ethercat-network-topology/>

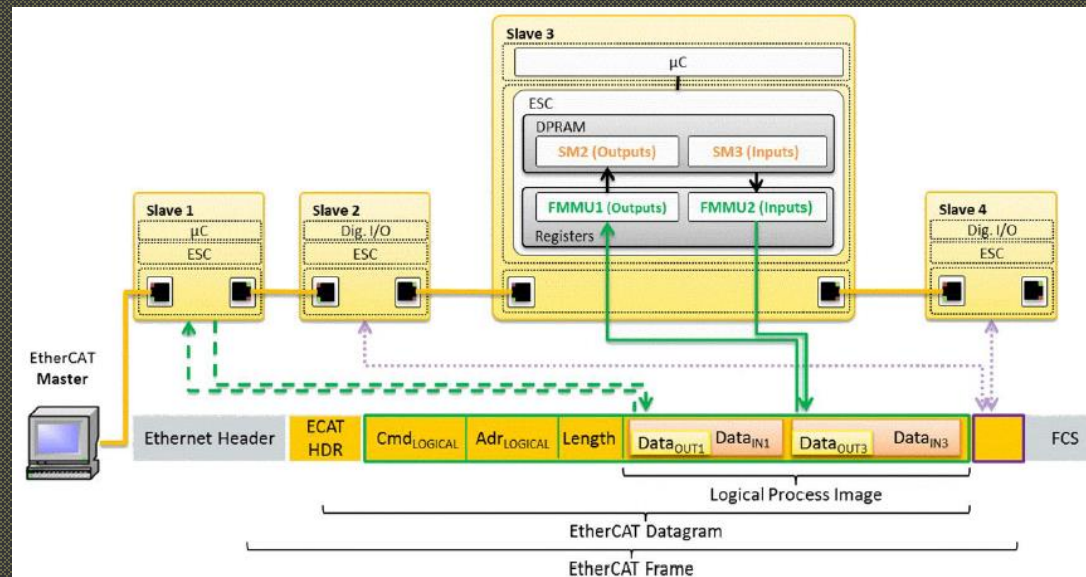


9A.2 The Introduction to EtherCAT Fundamentals

EtherCAT improves Ethernet efficiency by eliminating the need for each device to fully receive and process packets before passing them on. Instead, each device reads or inserts its data on the fly as the message (telegram) passes through. This is possible thanks to a special hardware component called the FMMU (Fieldbus Memory Management Unit).

- Unlike other Ethernet systems, EtherCAT devices do not need to receive, interpret, and resend frames one by one.
- Devices read/write only the part of the data relevant to them while the frame moves through - delays are only a few ns.
- EtherCAT uses the full 100 Mbit/s bandwidth in both directions, leading to effective throughput over 100 Mbit/s.

- Typically, a single frame contains data for multiple slaves (if not the entire network).
- Each slave has a FMMU to extract data from the packet and map the logical addresses to the physical addresses in the ESC.



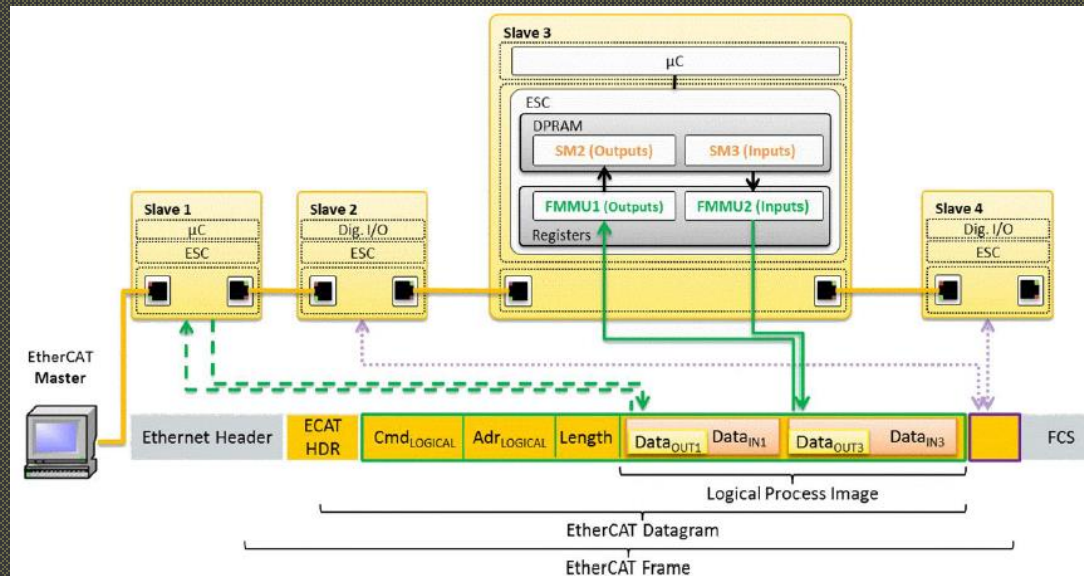
9A.2 The Introduction to EtherCAT Fundamentals

1. EtherCAT Master

- The master (on the left) sends out one Ethernet frame that contains data for all devices on the network.
- This frame includes the Ethernet header and EtherCAT datagram, the part that holds all commands and data.

2. EtherCAT Frame

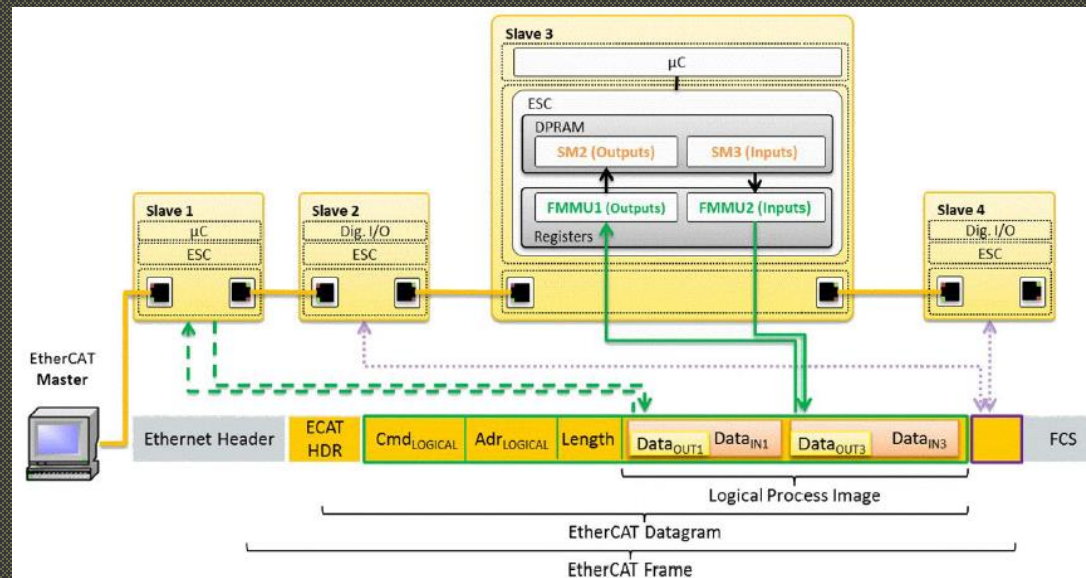
- The frame travels through all the slave devices in sequence (Slave 1 → Slave 2 → Slave 3 → Slave 4).
- It is like a train carrying many boxes (datagrams). Each slave opens only its own box, adds or reads its data, and passes the train along - without stopping it.



9A.2 The Introduction to EtherCAT Fundamentals

3. Slaves (Devices)

- Each slave has an EtherCAT Slave Controller (ESC); this is the small chip that handles EtherCAT communication.
- The ESC allows the slave to:
 - Read input data meant for the master.
 - Write output data meant for actuators or drives.
- Inside each slave, there are:
 - Sync Managers (SM) - handle data movement between the master and slave.
 - Fieldbus Memory Management Units (FMMU) - map the data correctly into memory areas.
- Example in the diagram:
 - Slave 3 reads its input data (SM3) and writes its output data (SM2) into the EtherCAT frame as it passes by.



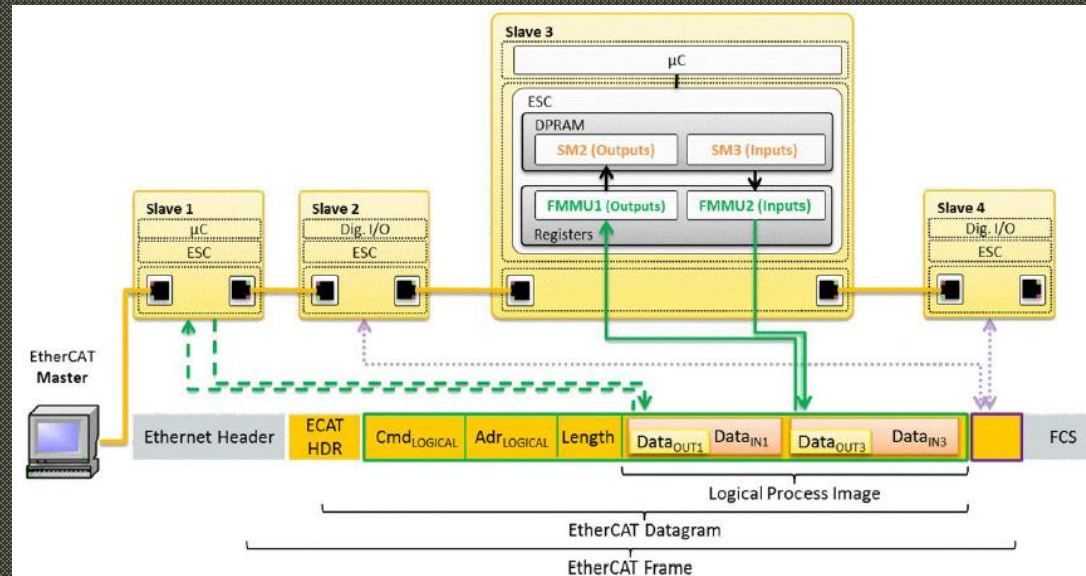
9A.2 The Introduction to EtherCAT Fundamentals

4. Logical Process Image

- The EtherCAT frame holds a logical process image - basically, one big block of memory that contains all inputs and outputs from all devices.
- This means the master sees the whole system as one continuous memory map - easy and fast to update.

5. FCS (Frame Check Sequence)

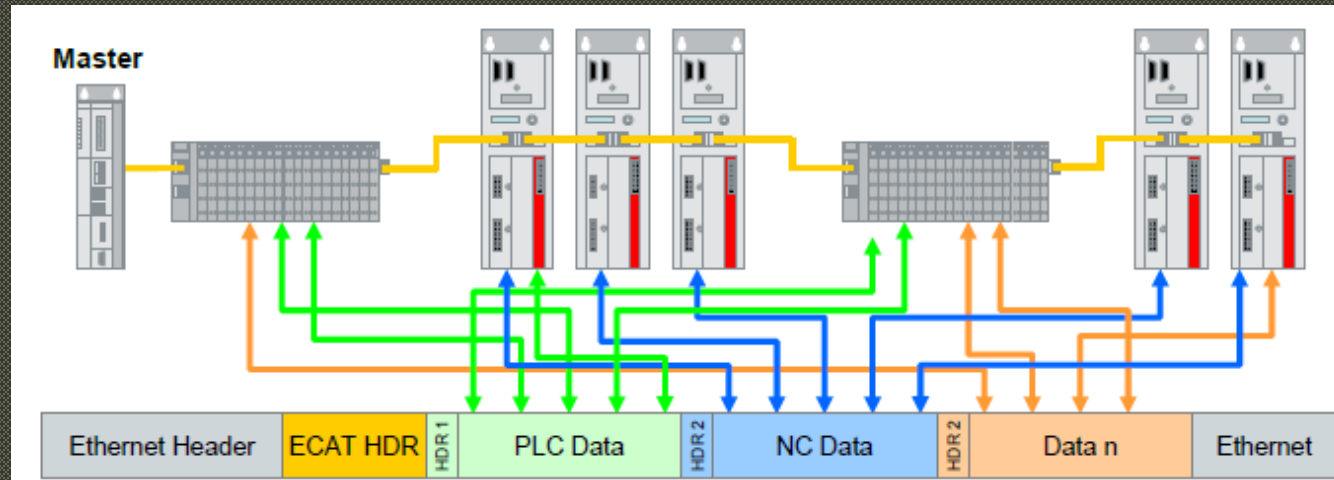
- This is the small section at the end of the frame used to check if the data is correct (error detection).



9A.2 The Introduction to EtherCAT Fundamentals

Since one frame serves many devices in both directions (sending and receiving), the usable data rate jumps to ~ 90%, compared to ~5% in traditional Ethernet real-time use. What takes up the remaining ~ 10%?

- **Frame Overhead:** Ethernet frames include headers and checksums for reliable delivery.
- **Protocol Structure:** Telegrams carry extra control and command data, not just process data.
- **Processing Delay:** Devices insert/extract data "on the fly," but even nanosecond delays add up.
- **Sync & Diagnostics:** Some bandwidth is used for clock sync, error checking, and control functions.
- **Full-Duplex Limits:** Despite using both send/receive channels (2×100 Mbps), only ~ 90% is usable for actual payload.



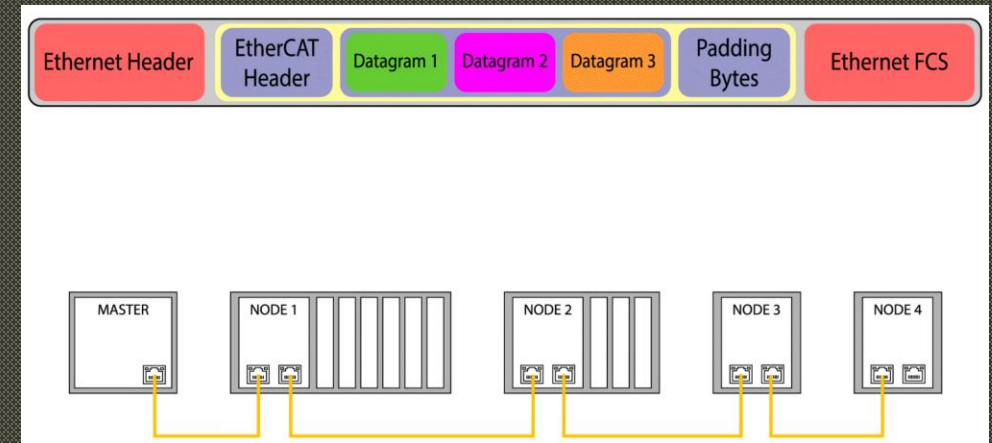
9A.3 EtherCAT Protocol Deep Dive

Introduction to EtherCAT Protocol

- EtherCAT protocol defines how the Master and Slaves communicate.
- Builds on standard Ethernet frames, but adds datagrams for on-the-fly processing.
- Focus: deterministic and real-time communication for automation.
- Analogy: Telegrams sent in a train (Ethernet frame), each station (Slave) reads/writes messages while the train moves.

EtherCAT Telegrams

- Telegram: the payload within an EtherCAT frame.
- Contains 1 or more datagrams.
- Components of a datagram:
 - Datagram header: type, length, command
 - Payload: actual data to read/write
 - Working Counter (WKC): confirms successful processing
- Frame passes all devices once; each device reads/writes on the fly.

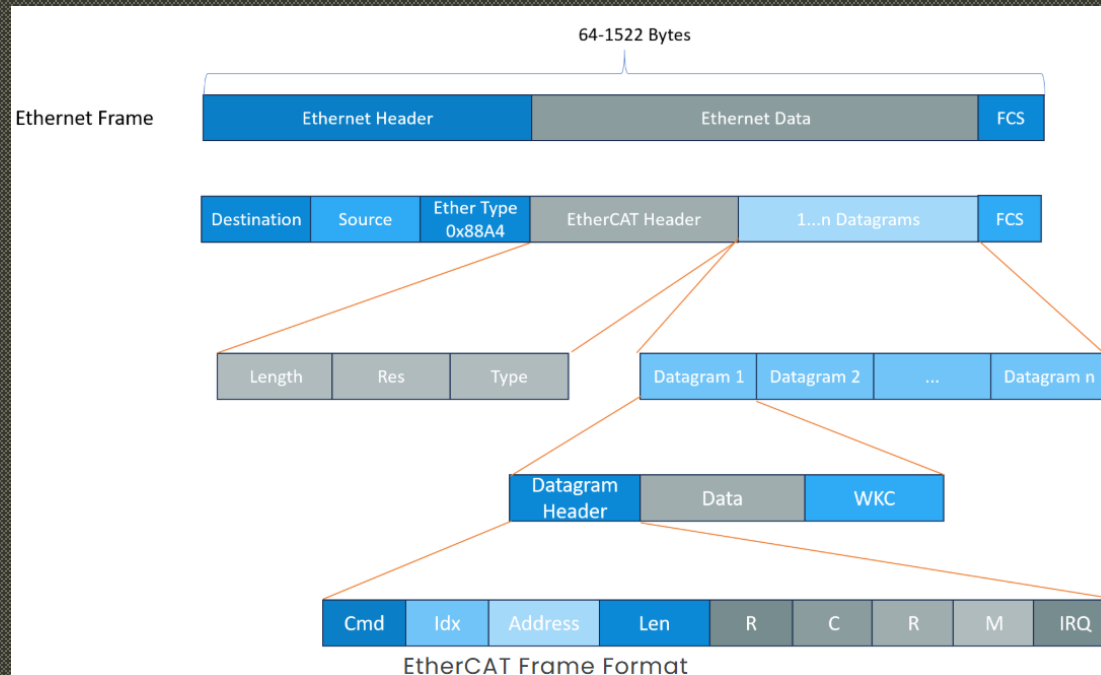
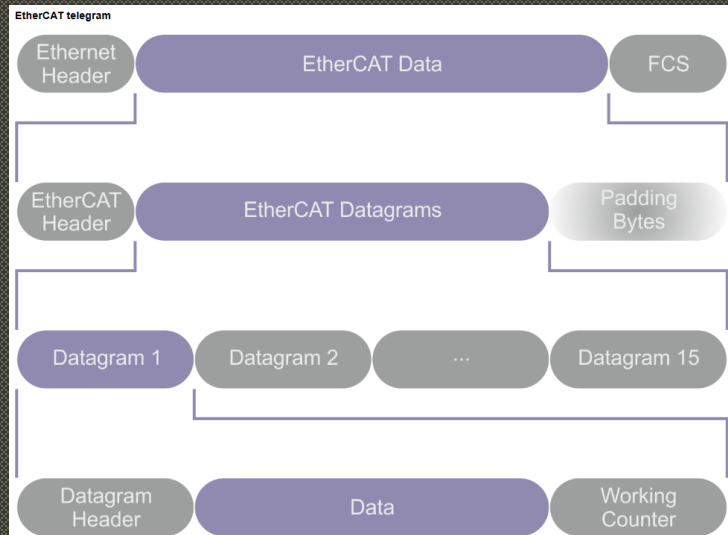


<https://www.acontis.com/en/what-is-ethercat-communication-protocol.html>

9A.3 EtherCAT Protocol Deep Dive

An EtherCAT telegram begins with an Ethernet header, followed by EtherCAT data, and ends with a Frame Check Sequence (FCS). The EtherCAT data section includes:

- An EtherCAT header
- Up to 15 datagrams, each containing:
 - A datagram header
 - Payload data (to be read or written)
 - A working counter
- If the total frame size is under 64 bytes, padding (1-32 bytes) is added to meet the minimum Ethernet frame size.



9A.3 EtherCAT Protocol Deep Dive

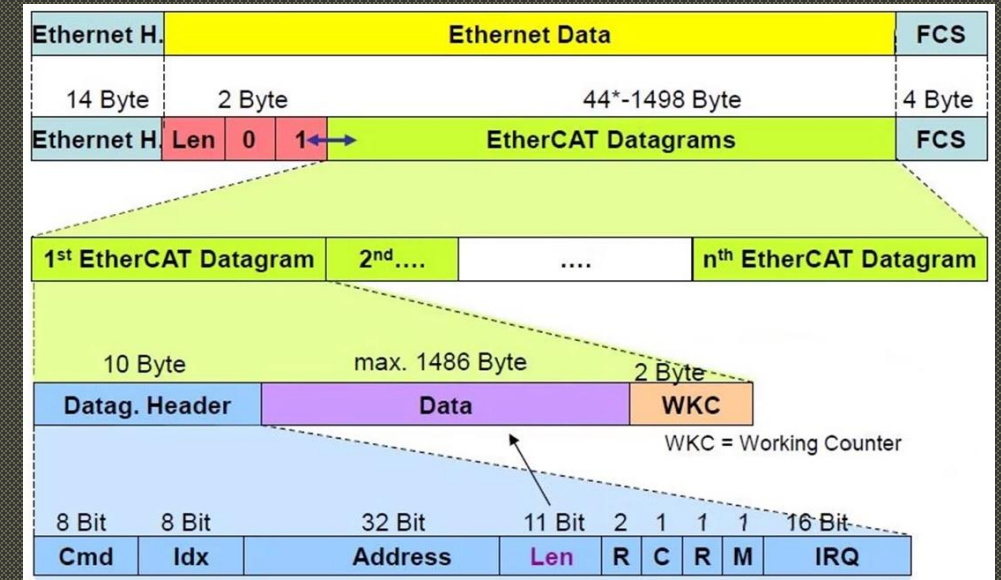
EtherCAT Header

The EtherCAT Header consists of three main components:

- a length specification,
- a reserved bit, and
- a protocol type specification.



Field	Data type	Value/Description
Length	11 bits	Length of the EtherCAT datagram (without FCS)
Reserved	1 bit	Reserved, 0
Type	4 bits	Protocol type. EtherCAT slave controllers (ESCs) only support EtherCAT commands (type = 0x1).



<https://www.ti.com/video/6174123090001#>

https://infosys.beckhoff.com/english.php?content=../content/1033/tc3_io_intro/1257993099.html&id=

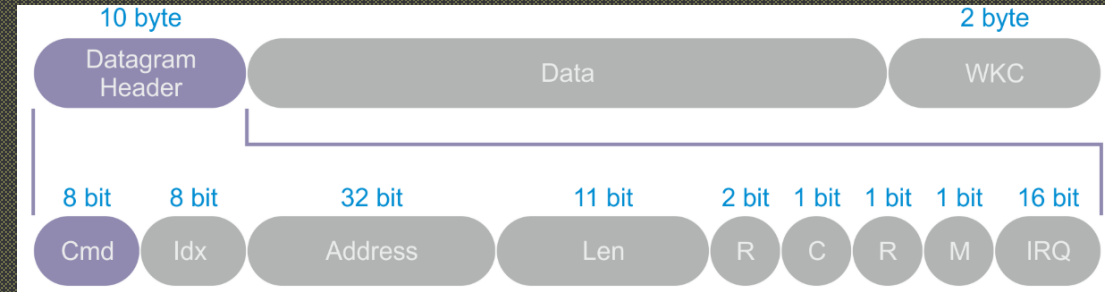
9A.3 EtherCAT Protocol Deep Dive

The Datagram Header includes the following elements:

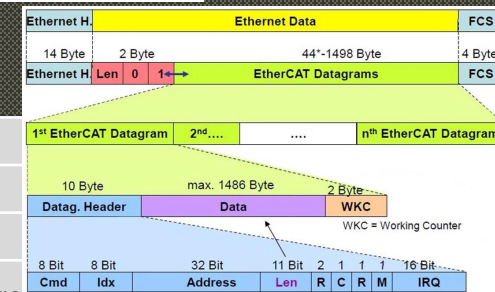
- EtherCAT Command Type: Specifies the type of command being executed.
- Numerical Identifier: A unique number assigned by the master to detect duplicate or lost datagrams.
- Address Specification: Defines the target address for the datagram.

This is followed by:

- Length Specification: Indicates the length of the data that follows in the datagram.
- Two Reserved Bits: Set aside for future use.
- Circulating Frame Prevention Bit: Prevents frames from looping endlessly.
- Another Reserved Bit: Reserved for potential future purposes.
- Next Datagram Bit: Indicates if another EtherCAT datagram follows.
- EtherCAT Event Request Register: Used to signal specific events within the network.



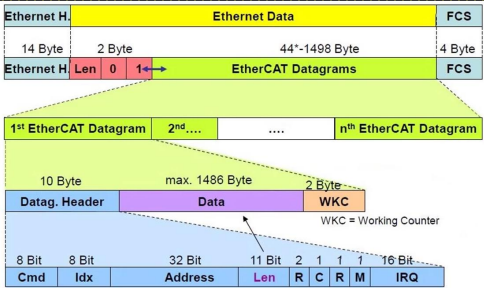
Field	Length (bits)	Description
Cmd	8	Command type
Idx	8	Index to identify frame
Address	32	Node Address concerning this datagram
Len	11	Length of the data in this datagram
R	3	Reserved, 0
C	1	Circulating frame (0 – Not circulating vs 1 being circulated)
M	1	Multiple EtherCAT datagrams (0 - Last datagram vs 1 - Another EtherCAT datagram follows)
IRQ	16	Event request register of all slave devices combined
Data	Variable	Data to be read or written
WKC	16	Working Counter



9A.3 EtherCAT Protocol Deep Dive

EtherCAT Command Types

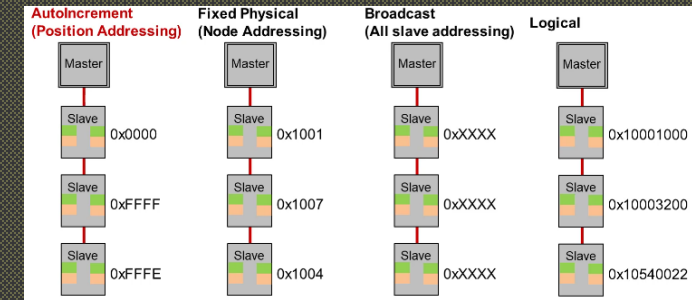
The table below lists EtherCAT command types. For combined read/write operations, the read occurs before the write.



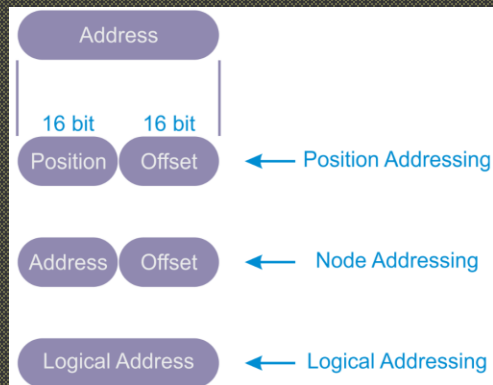
Cmd	Abbreviation	Name	Description
0	NOP	No Operation	A slave ignores the command.
1	APRD	Auto Increment Read	A slave increments the address. A slave writes the data it has read to the EtherCAT datagram if the address received is zero.
2	APWR	Auto Increment Write	A slave increments the address. A slave writes data to a memory area if the address received is zero.
3	APRW	Auto Increment Read Write	A slave increments the address. A slave writes the data it has read to the EtherCAT datagram and writes the newly acquired data to the same memory area if the received address is zero.
4	FPRD	Configured Address Read	A slave writes the data it has read to the EtherCAT datagram if its slave address matches one of the addresses configured in the datagram.
5	FPWR	Configured Address Write	A slave writes data to a memory area if its slave address matches one of the addresses configured in the datagram.
6	FPRW	Configured Address Read Write	A slave writes the data it has read to the EtherCAT datagram and writes the newly acquired data to the same memory area if its slave address matches one of the addresses configured in the datagram.
7	BRD	Broadcast Read	All slaves write a logical OR of the data from the memory area and the data from the EtherCAT datagram to the EtherCAT datagram. All slaves increment the Position field.
8	BWR	Broadcast Write	All slaves write data to a memory area. All slaves increment the Position field.
9	BRW	Broadcast Read Write	All slaves write a logical OR of the data from the memory area and the data from the EtherCAT datagram to the EtherCAT datagram; all slaves write data to the memory area. BRW is typically not used. All slaves increment the Position field.
10	LRD	Logical Memory Read	A slave writes data it has read to the EtherCAT datagram if the address received matches one of the FMMU areas configured for reading.
11	LWR	Logical Memory Write	Slaves write data to their memory area if the address received matches one of the FMMU areas configured for writing.
12	LRW	Logical Memory Read Write	A slave writes data it has read to the EtherCAT datagram if the address received matches one of the FMMU areas configured for reading. Slaves write data to their memory area if the address received matches one of the FMMU areas configured for writing.
13	ARMW	Auto Increment Read Multiple Write	A slave increments the Address field. A slave writes data it has read to the EtherCAT datagram when the address received is zero, otherwise it writes data to the memory area.

9A.3 EtherCAT Protocol Deep Dive

- Position Addressing used by master only during start-up to scan the network:
 - Position (16-bit) → identifies where the slave is in the network (its order in the chain).
 - Offset (16-bit) → identifies which part of the data in that slave to access.
- Node Addressing – Fixed Physical Addressing
 - Master uses the slave's unique address instead of its position in the chain.
 - Still includes an offset to indicate where inside that node's data to read or write.
 - This is useful when the physical order of slaves may change, but their addresses stay the same.
- Broadcast Addressing
 - Used to communicate with all slaves simultaneously, typically during initialization.
- Logical Addressing
 - The EtherCAT master treats the entire system's data as one big continuous memory block - the “process image”.
 - ”Each piece of data (input/output) has a fixed logical address inside that image.
 - Makes programming and data handling much easier, since you don't worry about which device it comes from.



<https://www.ti.com/video/6174123090001#>

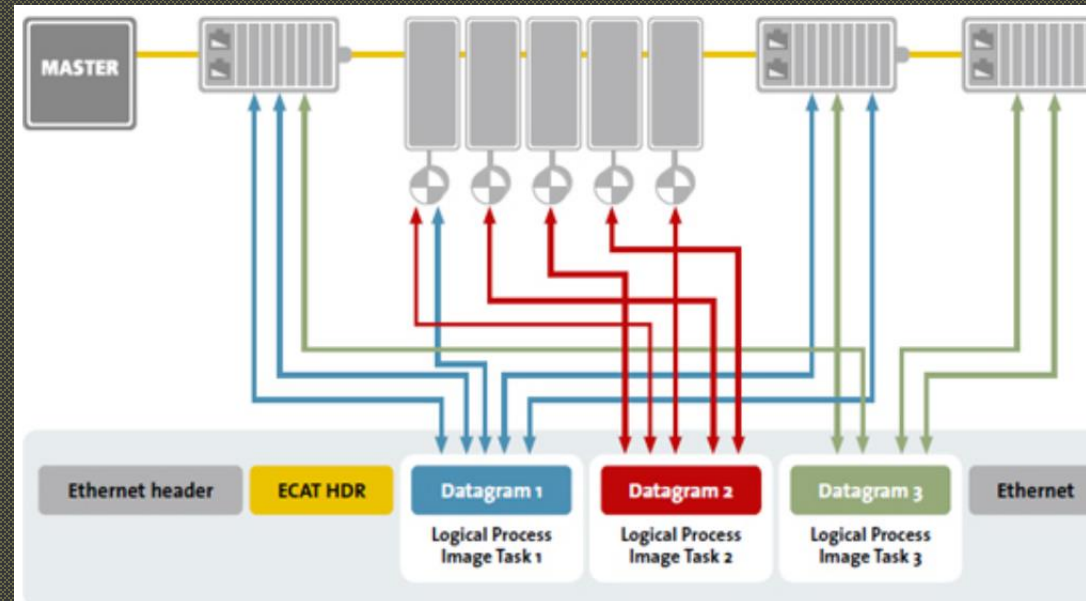


Mode	Field	Data type	Value/Description
Position address / auto-increment address	Position	WORD	Each slave increments the position value. The slave is addressed if the position is 0.
	Offset	WORD	Local register address or local memory address of the ESC
Node address / configured station address and configured station alias	Address	WORD	The slave is addressed if its address corresponds to the Configured Station Address or the Configured Station Alias (if enabled).
	Offset	WORD	Local register address or local memory address of the ESC
Broadcast	Position	WORD	Each slave increments the Position field (which is not used for addressing).
	Offset	WORD	Local register address or local memory address of the ESC
Logical address	Address	DWORD	Logical address (configured by the FMMUs) The slave is addressed if the FMMU configuration corresponds to the Address field

9A.3 EtherCAT Protocol Deep Dive

EtherCAT uses primarily logical addressing to exchange data quickly between the MainDevice (Master/Controller) and many SubDevices (Slaves) like sensors, drives, and I/O modules.

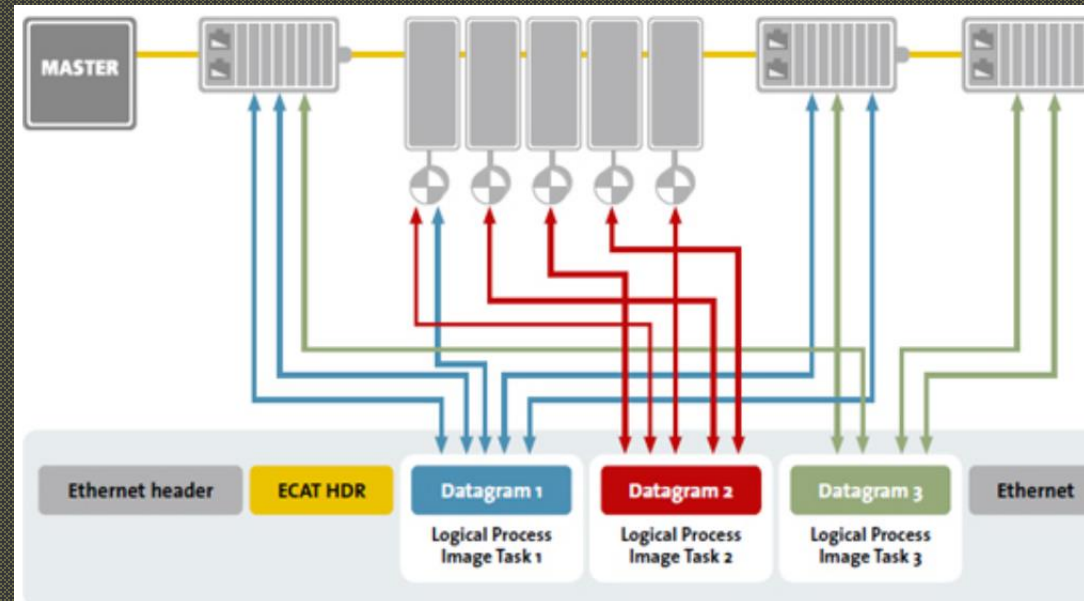
- Each SubDevice is assigned a location in a big shared memory space.
- Several devices can share the same memory area, so one message (called a datagram) can talk to multiple devices at once.
- The controller decides when and what data to send or read - for example:
 - Fast updates for motion control
 - Slower updates for basic I/O



9A.3 EtherCAT Protocol Deep Dive

After verifying the network configuration, the MainDevice assigns fixed addresses to each node, enabling stable communication even if the network topology changes.

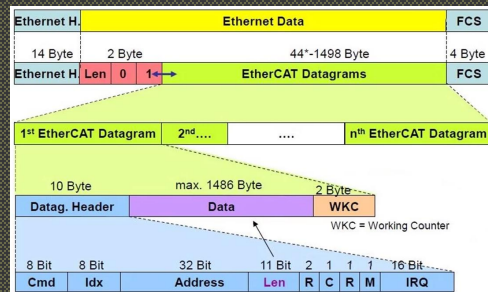
- There are two ways for SubDevices to communicate:
 - Direct SubDevice-to-SubDevice communication (only downstream) – fast but depends on fixed topology.
 - Indirect communication via the MainDevice – more flexible, uses two bus cycles, but still faster than other technologies due to EtherCAT's high performance.



9A.3 EtherCAT Protocol Deep Dive

The Working Counter (WKC) is the number of interactions contained in a given datagram. It is used to verify successful communication in EtherCAT. It is incremented by each device that:

- Is successfully addressed, and
- Successfully performs a read, write, or read/write operation.
- Each datagram has an expected WKC value, and the master compares this with the actual value after the telegram completes its path. If the values match, the operation was successful. If not, it signals an error or missed device.



- **Working Counter (WKC)**
 - WKC = success confirmation mechanism
 - Each Slave increments WKC if it processes the datagram correctly
 - Master reads WKC at the end of the frame to ensure data integrity

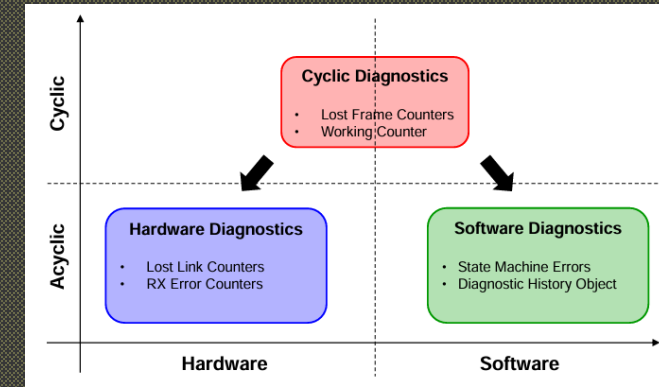
Example:

- 3 Slaves, Master sends APRD command
- Each Slave increments WKC by 1 → Master expects WKC = 3
- If WKC < 3 → indicates error or missing device

Command	Success	Increment
Read command	No success	No change
	Reading successful	+1
Write command	No success	No change
	Writing successful	+1
Read/write command	No success	No change
	Reading successful	+1
	Writing successful	+2
	Reading and writing successful	+3

9A.4 EtherCAT Diagnostics

EtherCAT networks, like other fieldbus systems, are susceptible to errors that can be categorized into two main types: hardware and software.



<https://www.ti.com/video/6174123090001#>

1. Hardware Errors

These are faults related to the physical components of the network.

- **Communication Failure:** The physical network connection is broken, preventing frames from reaching all devices or returning to the master. This can be caused by damaged cables, loose connections, or a slave device resetting unexpectedly.
- **Data Corruption:** Frames reach all devices, but electromagnetic interference (EMI) or faulty hardware corrupts the data, leading to errors in the bit sequence.

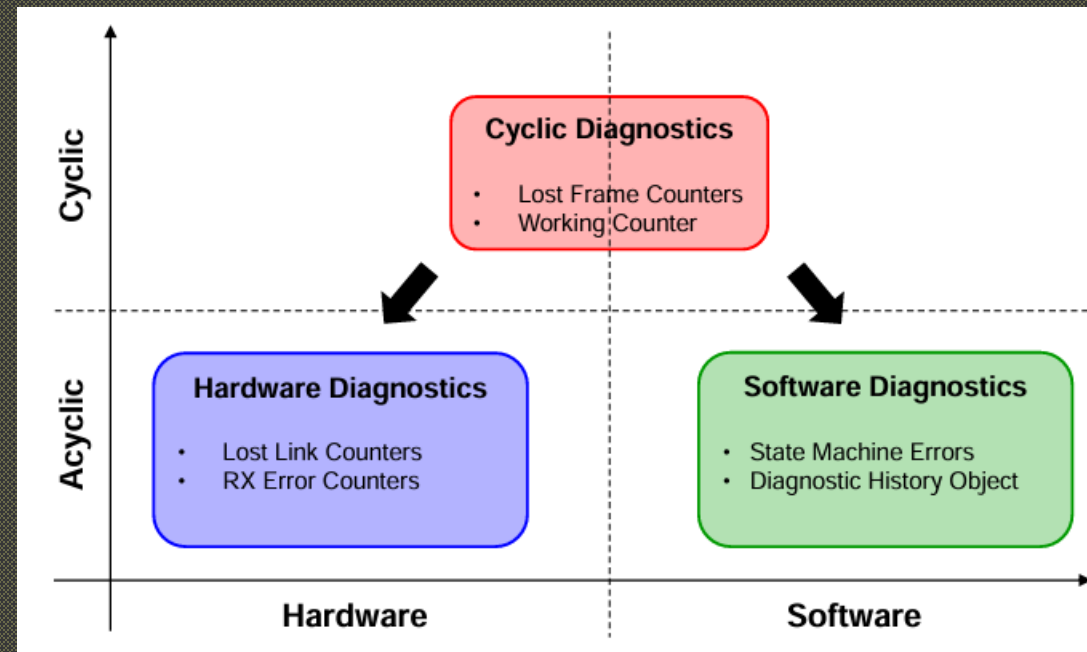
2. Software & Configuration Errors

These are faults related to the setup, parameters, or internal logic of the devices.

- **Initialization Errors:** During startup, the master provides incorrect configuration parameters that do not match a slave's capabilities. Examples include wrong process data size, invalid settings, or an unsupported cycle time.
- **Runtime Errors:** A slave that was functioning correctly encounters an error during normal operation. Common causes include loss of synchronization with the master or the expiration of an internal watchdog timer.

9A.4 EtherCAT Diagnostics

EtherCAT provides extensive diagnostic information for both hardware and software. This information can be classified as follows for simplicity:

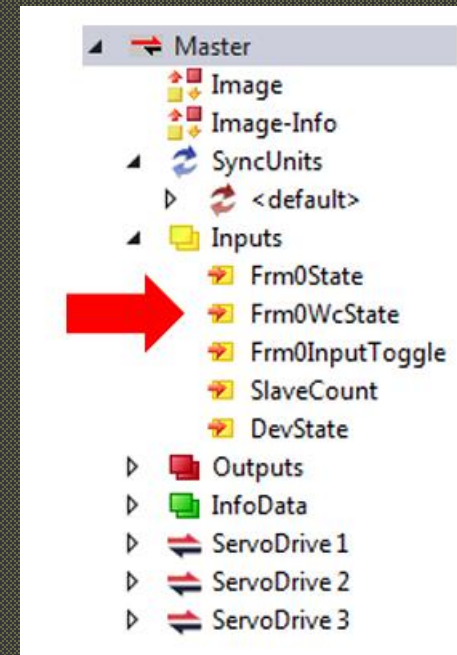
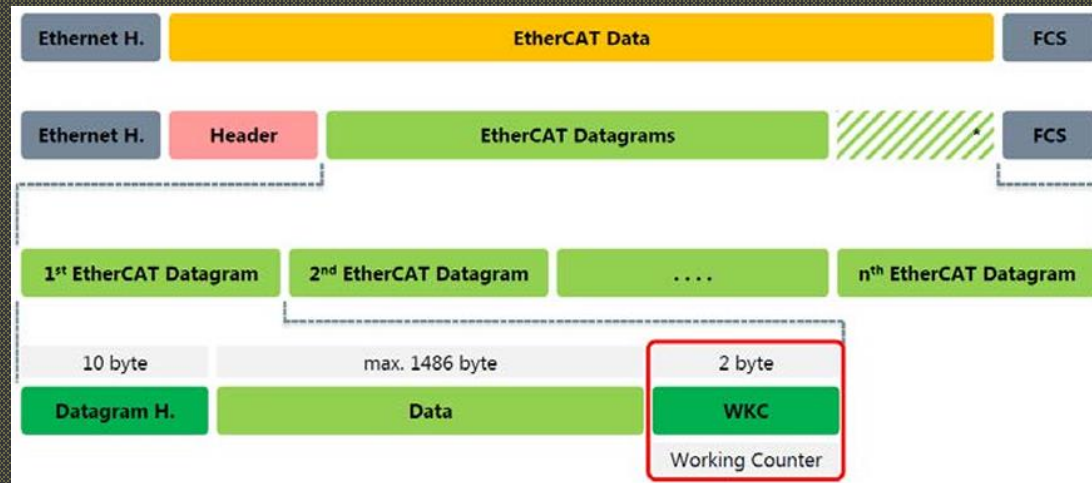


<https://www.ti.com/video/6174123090001#>

9A.4 EtherCAT Diagnostics

Cyclic Diagnostic

- Working Counter (WKC): A 16-bit field in each datagram that is incremented by every slave the datagram addresses.
- Error Handling: If a datagram returns with an unexpected WKC value, the master discards its input data.
- Status Monitoring: Master devices can optionally report the WKC status to the control application (e.g., PLC) via a cyclic variable in the process image (common for cyclic process data).

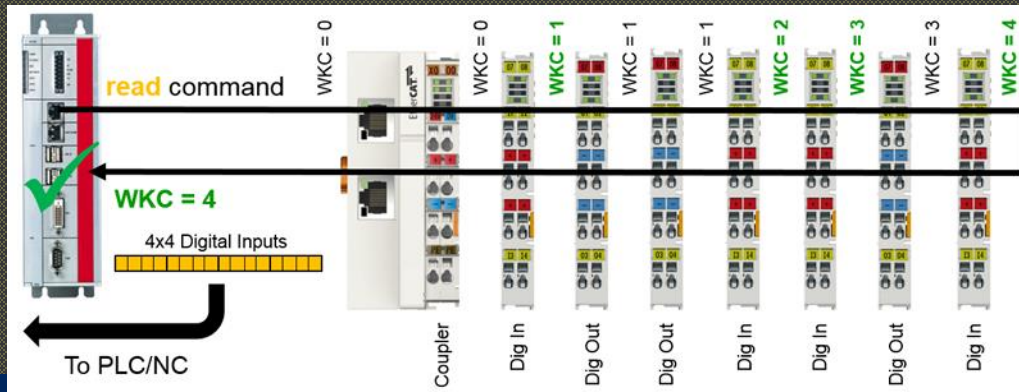


9A.4 EtherCAT Diagnostics

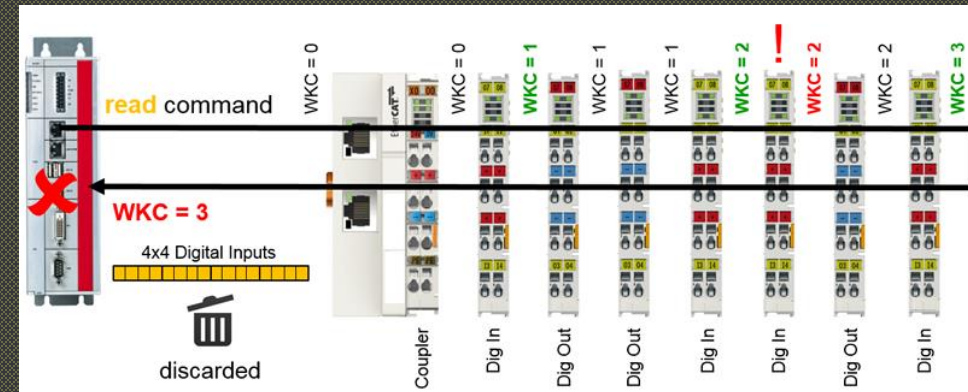
Cyclic Diagnostic

- Valid Data (WKC = Expected):
 - All slaves processed the datagram correctly.
 - The master forwards the new input data to the PLC/NC.
- Invalid Data (WKC $\neq$ Expected):
 - One or more slaves failed to process the datagram.
 - The master discards the new input data.
 - The PLC/NC continues using the previous (old) data.

All addressed slaves (Digital Inputs, in the example below) successfully process the datagram.



One addressed slave (Digital Input, in the example below) fails to process the datagram.

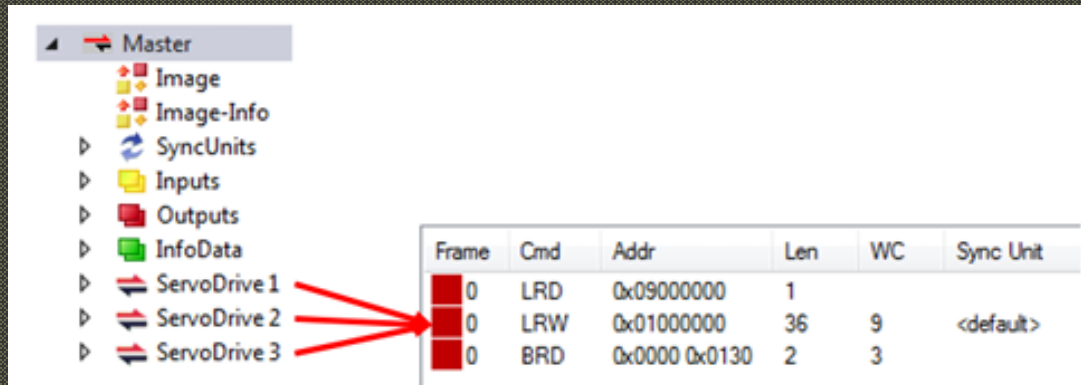


9A.4 EtherCAT Diagnostics

Cyclic Diagnostic

- Masters can optionally enable to group network slaves into separate subsets called SyncUnits.

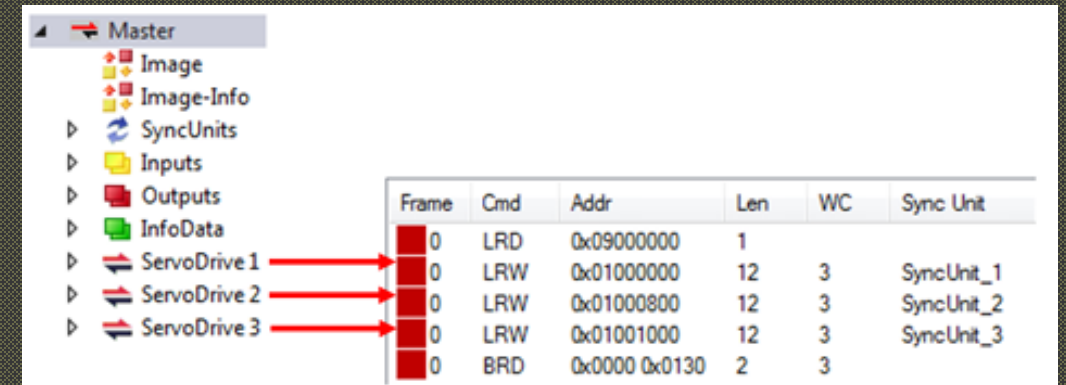
- One Default Sync Unit (The "All-or-Nothing" Approach)
 - All three drives are on the same list.
 - If one drive fails, its checkmark is missing.
 - Result: The master throws away the data from all three drives because the overall check is incomplete.



The screenshot shows the 'Master' configuration tree on the left with 'SyncUnits' expanded. Three red arrows point from 'ServoDrive 1', 'ServoDrive 2', and 'ServoDrive 3' to a single table entry, indicating they are all part of the same default SyncUnit.

Frame	Cmd	Addr	Len	WC	Sync Unit
0	LRD	0x09000000	1		
0	LRW	0x01000000	36	9	<default>
0	BRD	0x0000 0x0130	2	3	

- Separate Sync Units (The "Isolate the Problem" Approach)
 - Each drive is on its own separate list.
 - If one drive fails, only its own checkmark is missing.
 - Result: The master only throws away the data from the one faulty drive. The other two drives keep working with their good data.



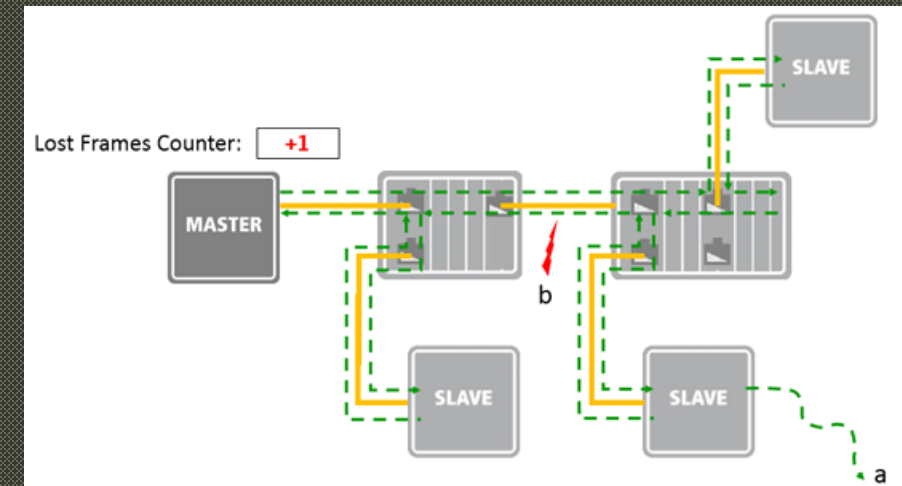
The screenshot shows the 'Master' configuration tree on the left with 'SyncUnits' expanded. Three red arrows point from 'ServoDrive 1', 'ServoDrive 2', and 'ServoDrive 3' to three separate table entries, indicating they are each in their own SyncUnit.

Frame	Cmd	Addr	Len	WC	Sync Unit
0	LRD	0x09000000	1		
0	LRW	0x01000000	12	3	SyncUnit_1
0	LRW	0x01000800	12	3	SyncUnit_2
0	LRW	0x01001000	12	3	SyncUnit_3
0	BRD	0x0000 0x0130	2	3	

9A.4 EtherCAT Diagnostics

Hardware Diagnostic

1. Slaves have built-in error counters.
 - These counters are stored at standard memory locations within each slave device.
 - The master can read these counters and make the data available to the control application (e.g., a PLC program).
2. The master tracks "Lost Frames."
 - A frame is considered "lost" if:
 - (a) It never returns to the master.
 - (b) It returns but is corrupted and unreadable.
 - The master counts these events with a Lost Frame Counter.
3. How to use these diagnostics:
 - The Lost Frame Counter on the master is the first alarm for hardware issues (e.g., bad cables, noise).
 - If this counter increases, you should then check the individual error counters on the slave devices to find the root cause.



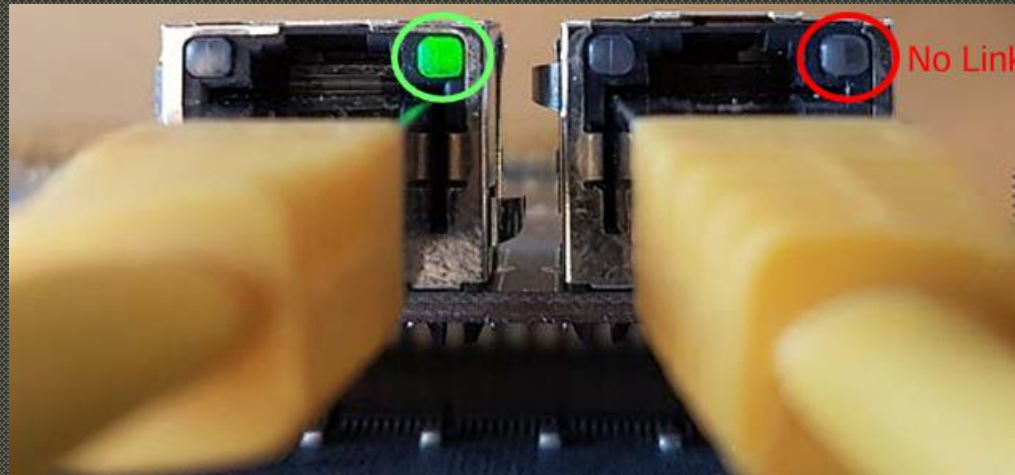
<https://www.ti.com/video/6174123090001#>

9A.4 EtherCAT Diagnostics

Hardware Diagnostic

1. First, A Visual Check is required: The Link/Activity LED

- Every EtherCAT slave with removable connectors has a Link/Activity LED for each port.
- Simple Rule: If the physical link is broken, the LED will be permanently off.
- Action: Before checking complex counters, always look at the LEDs. A dark LED means "No Link!" and points to a hardware break.



9A.4 EtherCAT Diagnostics

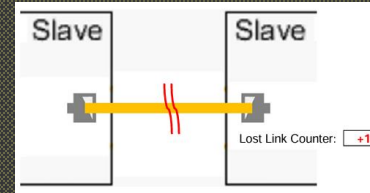
Hardware Diagnostic

2. Understanding the Key Error Counters

Main hardware error counters and what they indicate:

a. Lost Link Counter

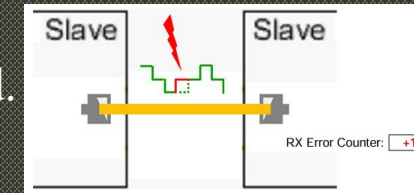
- What it means: The physical communication channel was interrupted. No frames could be sent to the next device.
- Most Likely Causes:
 - Device lost power or was reset.
 - Damaged cables, connectors, or poor contacts.
 - Severe Electromagnetic Interference (EMI).



<https://www.ti.com/video/6174123090001#>

b. RX (Receive) Error Counter

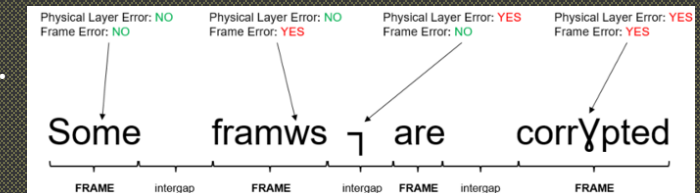
- What it means: The hardware signal was corrupted, so the data was discarded.
- Most Likely Causes:
 - Sporadic Increments: Usually caused by external EMI.
 - Rapid, Systematic Increments: Usually a sign of damaged devices or cabling.



<https://www.ti.com/video/6174123090001#>

c. Invalid Frame Counter

- What it means: This is a combination of two more specific errors:
 - Physical Layer Errors: Individual "symbols" in the signal were invalid. This can happen inside or outside of a data frame.
 - Frame Errors: The overall sequence of bits in a frame was corrupted (this can only happen inside a frame).



9A.4 EtherCAT Diagnostics

Software Diagnostic

The EtherCAT State Machine

- Every slave device operates according to a defined state machine.
- The master commands state transitions to control the slave's functionality.

State	Communication Capabilities	Description
Init	None	The starting state. No data exchange is possible.
Pre-OP	Acyclic (Mailbox) only	Configuration data can be transferred, but not cyclic process data.
SafeOP	Acyclic & Cyclic (locked)	Process data exchange is active, but output data is held in a safe state.
OP	Acyclic & Cyclic (fully enabled)	Full operational state. All communication is active without restrictions.
Boot	Firmware Update (via Mailbox)	Optional state used only for updating device firmware.

State	It's Like...	What It Can Do
Init	Park (P)	The engine is on, but you can't drive or even use the radio.
Pre-OP	Neutral (N)	You can use the radio (acyclic data), but you still can't drive.
SafeOP	Drive (D) with Parking Brake On	The car is in gear, but the parking brake is locked. The engine is ready, but the wheels won't turn.
OP	Driving Normally	You're on the road! Everything works perfectly.
Boot	At the Mechanic	The car is only there to get a software update.

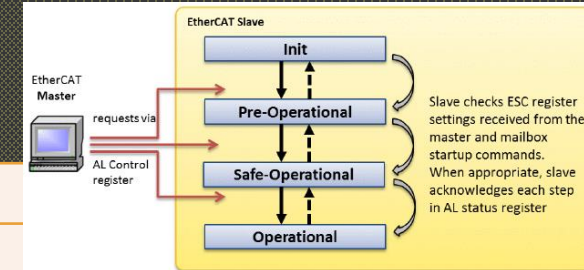
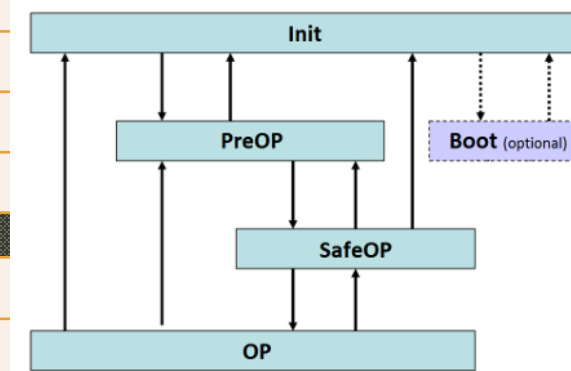


Figure 11: EtherCAT Network Initialization



<https://www.ti.com/video/6174123090001#>

9A.4 EtherCAT Diagnostics

Software Diagnostic

How State Control Works

- The master controls a slave's state by writing to its Application Layer (AL) Control register (0x0120).
- Each slave reports its current state and any errors in its AL Status register (0x0130).
- A slave can only go backwards to a lower state on its own if it detects an internal error.

Visual Diagnostics: The Run LED

Slaves with removable connectors have a Run LED that visually indicates their current state.



State	Run LED Indication
Init	Off
Pre-OP	Blinking Slowly
SafeOP	Single Flash with Long Pause
OP	On (Steady)
Boot	Flickering Fast or Off

<https://www.ti.com/video/6174123090001#>

9A.4 EtherCAT Diagnostics

Software Diagnostic

Visual Diagnostics: The Error LED

Many slaves also feature an Error LED (or a combined Status LED) to report specific issues.

Error Condition	Error LED Indication
No Error	Off
Configuration Error	Blinking
Generic Runtime Error	Single Flash
Process Data Watchdog Expired	Double Flash



<https://www.ti.com/video/6174123090001#>

Combining Visual and Digital Diagnostics

- If a slave cannot reach the state requested by the master, it reports an error in the AL Status register and writes a specific error code in the AL Status Code register (0x0134).
- The master can read this code to get a precise diagnostic message, which complements the visual information from the LEDs.

Errors in the EtherCAT state machine fall into two clear categories, depending on when they happen.

Category	When It Happens	What Goes Wrong	Common Error Examples
1. Initialization Error	During system start-up, before full operation.	The master commands the slave to enter OP state, but the slave refuses because its setup is incorrect or invalid.	• 0x0003 Invalid Device Setup • 0x001D Invalid Output Configuration • 0x0035 Invalid Sync Cycle Time
2. Runtime Error	During normal operation, after start-up is complete.	The slave spontaneously leaves OP state and goes back to a safer state (like SafeOP) because it detected a problem.	• 0x001A Synchronization Error • 0x001B Sync Manager Watchdog • 0x002C Fatal SYNC Error

9A.5 EtherCAT Communication Profiles

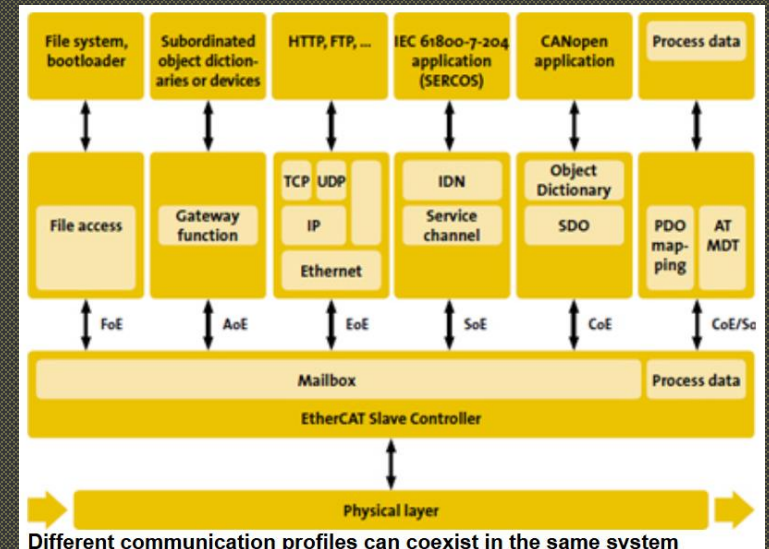
EtherCAT can configure and diagnose devices using special messages (called acyclic communication) that are reliable and automatically corrected if there's an error.

To support different device types, EtherCAT uses communication profiles, like:

- CoE – for CAN-based devices
- SoE – for servo drives
- EoE – to send regular Ethernet data
- FoE – for file transfers
- AoE – for Beckhoff ADS communication

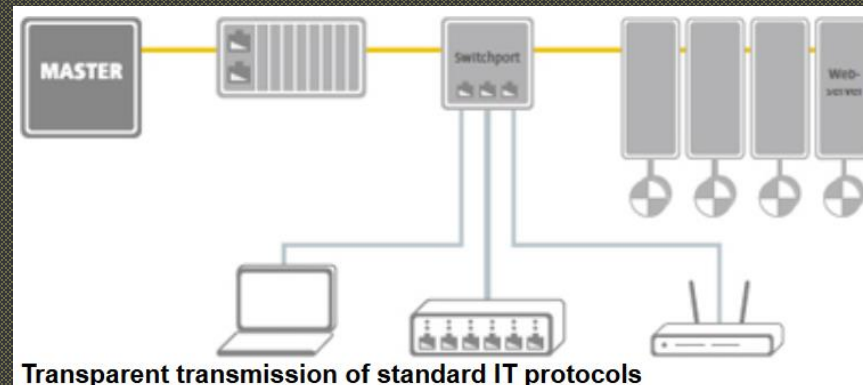
Two Types of Data Transfer:

- Cyclic (Real-Time)
 - Sent on every communication cycle
 - Fast, for control data like sensor/actuator values
- Acyclic (As Needed)
 - Sent only when required (e.g., configuration, diagnostics)



9A.5 EtherCAT Communication Profiles

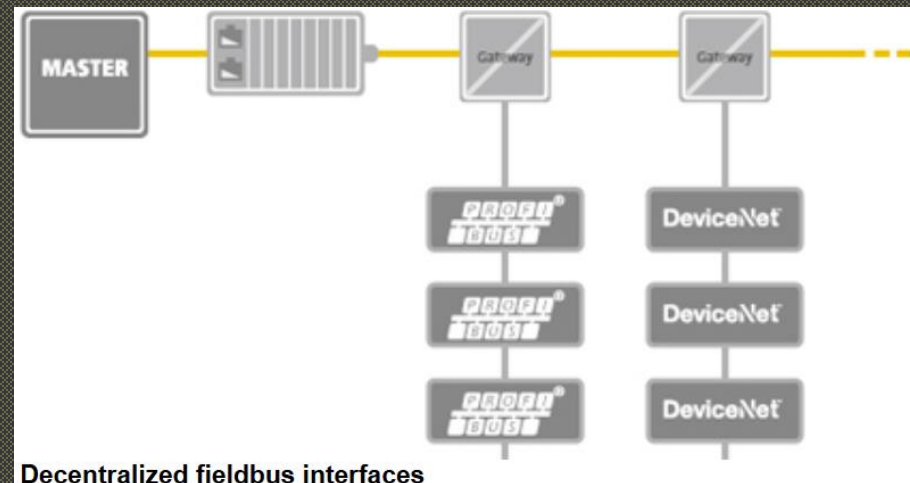
- CoE (CANopen over EtherCAT) work like CANopen, making it easy to upgrade old devices. It supports faster data, bigger messages, and reuses existing CANopen profiles and firmware.
- SERCOS™ is a real-time motion control standard; its servo drive profile (IEC 61800-7) can be used with EtherCAT via the EtherCAT Mailbox for parameter access (SoE).
- EtherCAT can tunnel standard Ethernet (TCP/IP) traffic using the EoE protocol, allowing Ethernet devices and services (like HTTP, FTP, etc.) to work within an EtherCAT network without affecting real-time performance.
- FoE (File over EtherCAT) is a lightweight protocol like TFTP that allows file access and firmware updates without needing a TCP/IP stack, making it ideal for bootloaders.
- ADS (Automation Device Specification) over EtherCAT (AoE) is a lightweight, Mailbox-based protocol used for flexible, routable, and parallel communication - ideal for accessing sub-networks like CANopen® or IO-Link™ through gateways, and for remote diagnostics with minimal overhead.



9A.5 EtherCAT Communication Profiles

EtherCAT's high bandwidth allows older fieldbus systems to be integrated via gateways, enabling a gradual upgrade without replacing existing devices.

- These gateways reduce the need for extra hardware like extension cards, allowing for more compact controllers.
- Data from the legacy system is seamlessly included in the EtherCAT process image, simplifying communication and system design.



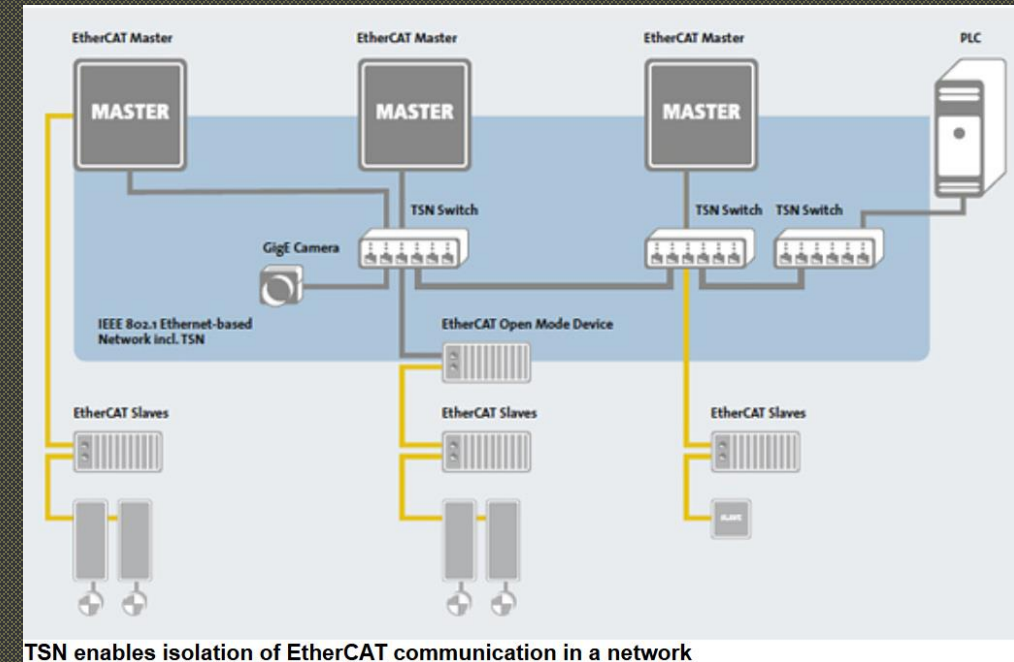
9A.5 EtherCAT Communication Profiles

Time-Sensitive Networking (TSN) is a set of IEEE 802.1 standards that improve Ethernet's real-time performance by adding features like:

- Time synchronization
- Scheduled traffic (reserved time slots for critical data)
- Frame preemption (pause low-priority data to send urgent messages)
- Stream reservation (guarantees bandwidth for specific data flows)

TSN allows real-time and regular traffic to coexist on the same network and supports multi-device communication across complex systems.

EtherCAT can integrate into TSN networks without modification, letting the MainDevice manage multiple EtherCAT segments with better flexibility and performance on standard Ethernet hardware.



TSN enables isolation of EtherCAT communication in a network

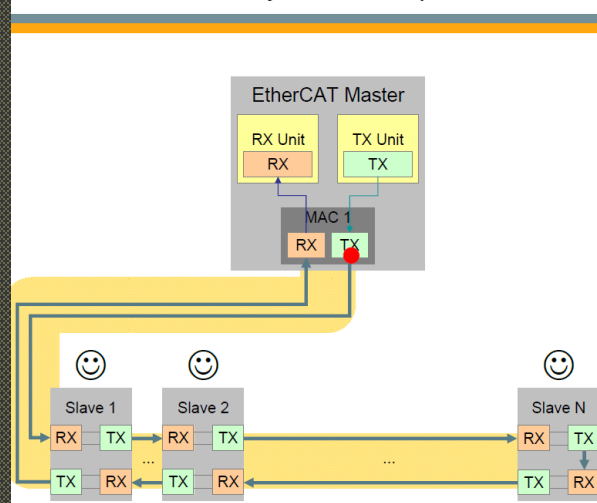
It requires:

TSN-capable Ethernet controller

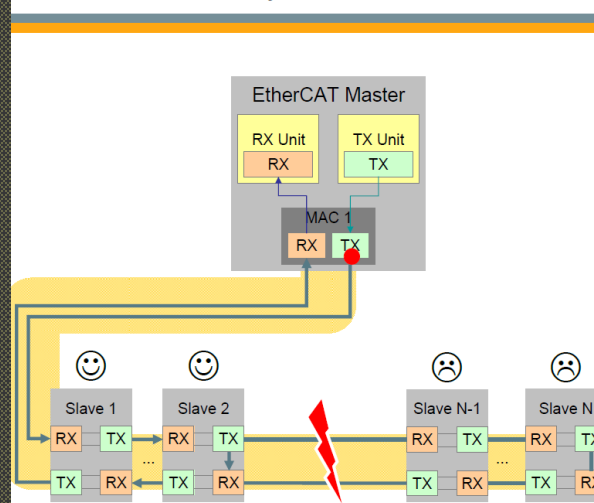
TSN-capable switches

9A.6 The EtherCAT Redundancy

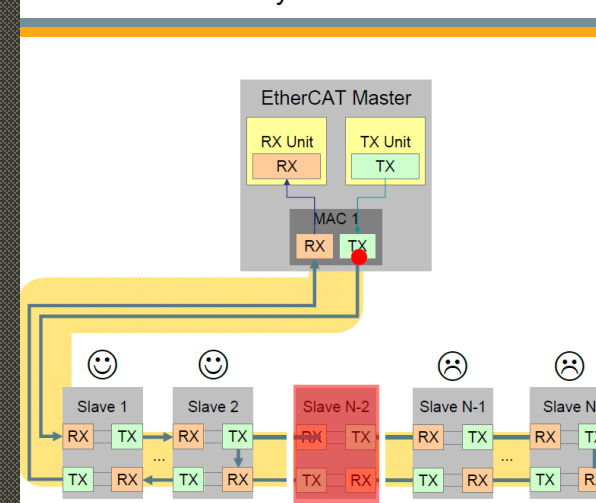
Without Redundancy: Normal Operation



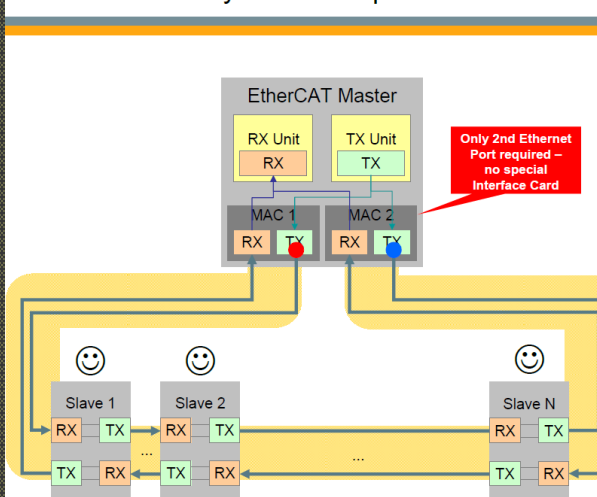
Without Redundancy: Cable Failure



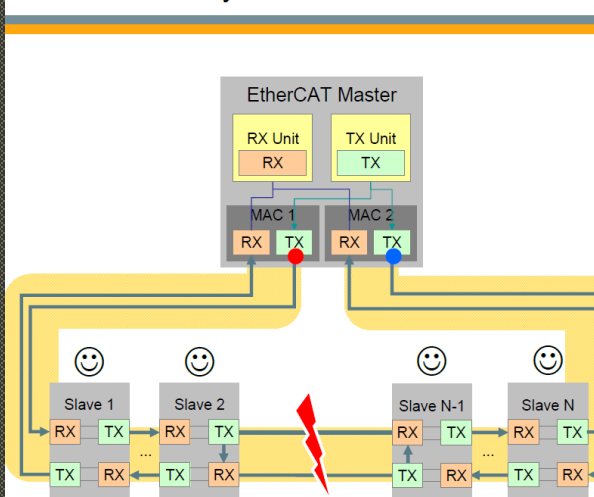
Without Redundancy: Node Failure



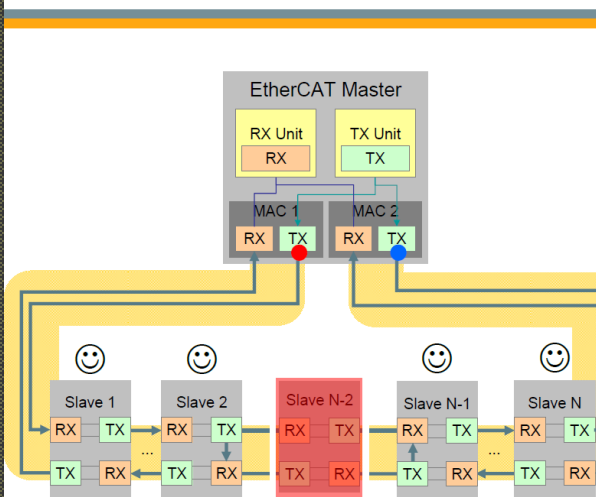
With Redundancy: Normal Operation



With Redundancy: Cable Failure



With Redundancy: Node Failure



9A.6 The EtherCAT Redundancy

Cable redundancy in EtherCAT is a feature that keeps the network running even if a cable breaks anywhere in the system. To do this, the network is arranged in a ring, and the master has two ports: one sends data forward, and the other sends data backward around the ring.

1. How it works in normal operation

- The master sends the same EtherCAT frames out of both ports at the same time.
- The “main” frame goes forward through all the slaves.
- The “redundancy” frame goes the opposite way but normally doesn’t change because slaves only process frames in the forward direction.
- Both frames eventually return to the master, and the master checks them.

2. What happens if a cable breaks

- A cable can break anywhere between any two slaves.
- When that happens, the forward frame can reach the slaves on one side of the break, and the reverse frame can reach the slaves on the other side.
- Each frame now contains part of the input data - so the master combines the two frames to rebuild the full data set.
- The working counters from both frames are added together to verify that all slaves were reached from one side or the other.

3. Why this is useful

- The network keeps running with no data loss, even with a single cable break.
- All EtherCAT communication continues normally - cyclic data (I/O updates) and mailbox messages.
- Once the cable is fixed, the network automatically returns to normal operation.

4. Limitations

- Redundancy can handle one cable break at a time.
- If more than one break occurs, all cables must be fixed before the network can tolerate another failure.

REFERENCES

- <https://www.ethercat.org/en/technology.htm>
- https://www.ethercat.org/en/ethercat_and_tsn.htm
- https://www.ethercat.org/download/documents/EtherCAT_Introduction_EN.pdf
- Slave Implementation Guide - Document: ETG.2200 V3.0.4

INDUSTRIAL NETWORKING

MENG 4010

Thank you!

Discussions?